UNIVERSITI TEKNOLOGI MARA

COMPARATIVE ANALYSIS OF SPEECH RECOGNITION MODELS WITH A FOCUS ON THE JAPANESE LANGUAGE

MUHAMMAD ALIFF AIMAN BIN ZOLKIFELI

MSc

January 2026

UNIVERSITI TEKNOLOGI MARA

COMPARATIVE ANALYSIS OF SPEECH RECOGNITION MODELS WITH A FOCUS ON THE JAPANESE LANGUAGE

MUHAMMAD ALIFF AIMAN BIN ZOLKIFELI

Dissertation submitted in partial fullfillment
of the requirements for the degree of
Master of Computer Science

College of Computing, Informatics and Mathematics

January 2026

ABSTRACT

Automatic speech recognition for formal Japanese remains challenging due to script variation and sensitivity to front-end processing. Additionally, comparative evidence across model families was limited under matched conditions. This study aimed to determine which ASR model family performed best for formal Japanese transcription. The preprocessing, feature extraction, and evaluation were standardized using same methods for all models. A Japanese TEDx dataset with manual subtitles was collected, cleaned, resampled to 16 kHz mono, segmented using VAD, filtered to remove low-quality or non-speaker segments. The collected data is split into 80/10/10 train/validation/test sets. Three model families were trained and evaluated on the same splits. The models evaluated included a Kaldi GMM–HMM pipeline (mono→tri3) with MFCC+CMVN, a SpeechBrain CRDNN–CTC system using log-Mel features with greedy and beam decoding, and fine-tuned Whisper variants from tiny to turbo. Performance was measured using CER as primary metric, WER, and RTF as secondary metric. Fine-tuned Whisper achieved the best accuracy, reaching 4.28% CER (turbo) and 4.22% WER (medium), while CRDNN–CTC achieved the fastest decoding with 0.0129 RTF and 15.23% CER. The best Kaldi stage (tri3) achieved 19.48% CER with 0.251 RTF. These results provided an actionable accuracy–latency trade-off guide for selecting Japanese ASR pipelines for formal transcription.

Keywords: Japanese ASR, formal speech transcription, GMM–HMM, CRDNN–CTC, Whisper fine-tuning

TABLE OF CONTENTS

LIST OF TABLES

LIST OF FIGURES

CHAPTER ONE
INTRODUCTION

1.1 Research Background

Human-computer interaction has evolved from manual input into more natural interfaces, and one of the natural interfaces is Automatic Speech Recognition (ASR). ASR has the capabilities to convert spoken language into text, which is useful in applications such as captioning, meeting minutes, media archiving, and voice-enabled search (W. Xu et al., 2023). In the context of the Japanese language, the quality of the transcription is still a challenge because of the multiple writing systems, which are kanji, hiragana, and katakana, and also context-sensitive readings that exist in the language (Koenecke et al., 2020).

Most ASR pipelines focus on the acoustic or end-to-end model (C. Xu et al., 2023). However, real-world performance relies on the preprocessing and feature extraction decisions made before the data is fed into the model. Latif et al. (2020) stated that the preprocessing and feature extraction step can significantly impact the accuracy and stability of the model performance. In addition to the preprocessing steps, the model's output also must be clean for easy reading, quoting, and storing. Ando and Fujihara (2021) stated that this step important for the usability of the transcripts.

This thesis focuses on formal Japanese speech transcription and compares three representative modeling approaches under controlled pre-processing and feature extraction. The first model is a GMM-HMM pipeline, which is a traditional model, the second model is a CRDNN-CTC model, which is a hybrid model based on a neural network, and the last model is a fine-tuned Whisper model, which is a transformer model. This thesis evaluates the model performance based on Character Error Rate (CER) as the primary metric, and Word Error Rate (WER) and Real-Time Factor (RTF) as secondary metrics.

1.2 Problem Statement

Despite rapid progress in ASR, the comparative performance of different model families for formal Japanese transcription remains unclear. Many studies have explored various architectures, including traditional GMM–HMM systems (Trmal, 2015), hybrid neural models like CRDNN–CTC (Chen et al., 2020), and large-scale transformer models such as Whisper (Radford et al., 2023). However, most studies focus on English or multilingual datasets, with relatively fewer investigations dedicated to Japanese (C. Xu et al., 2023). Of these, only a few compare models from different model families. Getting a clear comparison between these model families is also challenging as different studies use different datasets and preprocessing steps. As a result, it is difficult to draw definitive conclusions about which model type performs best for formal Japanese transcription tasks.

This study addresses that gap by conducting a controlled comparison of three representative models. These include a traditional statistical GMM–HMM, hybrid CRDNN–CTC, and fine-tuned Whisper transformer models. A standardized preprocessing, feature extraction, and evaluation protocols were used. All models were trained and evaluated on the same formal Japanese speech dataset with matched preprocessing configurations such as segmentation, resampling, and CMVN, and the feature types such as MFCC and log-Mel. Then, the outputs were post-processed uniformly to ensure readability and standardization. Finally, the same evaluation metrics, CER, WER, and RTF, were computed consistently to quantify both accuracy and usability. This systematic comparison will clarifies which model family setup is most effective for formal Japanese transcription under matched conditions, filling a critical gap in the literature and informing best practices for ASR system design in Japanese contexts.

1.3 Research Objectives

1. To identify the key requirements for constructing a speech-to-text model within the context of the Japanese language.

2. To analyze speech-to-text models to determine the most effective preprocessing and training configurations to reduce CER, WER, and RTF in

Japanese language processing.

3. To evaluate the CER, WER, and the transcription latency using RTF of different speech-to-text models when transcribing Japanese language.

1.4 Research Questions

1. What are the key requirements for constructing a speech-to-text model within the context of the Japanese language?

2. What is the most effective pre-processing and training setup to reduce CER, WER, and RTF in Japanese language processing?

3. How do the different speech-to-text models compare in terms of CER, WER, and RTF within the context of the Japanese language?

1.5 Scope of Study

This study focuses on speech-to-text transcription of the Japanese language using only formal speech. Dialect speech will not be included in this study. The main focus of this study is to produce a readable and standardized text rather than detect dialects or classify speaking styles. For the preprocessing and feature extraction, this study focuses on segmentation, resampling, and normalization, and two feature families of MFCC and log-Mel filterbanks.

The models that were compared in this study are GMM-HMM, CRDNN-CTC, and fine-tuned Whisper. The evaluation metrics that were used in this study are Character Error Rate (CER) as the primary metric, and Word Error Rate (WER) and Real-Time Factor (RTF) as secondary metrics. For the text post-processing, this study focuses on normalizing the output for formal use by removing filler words and using consistent script conventions. Semantic editing and translation are out of scope for this study.

For the dataset, this study will only use formal Japanese speech with available transcripts. The data that were used is TEDx talks in the Japanese language from YouTube. The audio and transcript were downloaded using Rawat et al. (2023) tool.

1.6 Significance of Study

This study aims to address the lack of an effective speech-to-text solution that focuses on the Japanese language. Most of the developed models focus on the English language or a generic transcription model that is developed for multiple languages. Organizations that rely on accurate transcripts, like broadcasters, government agencies, and archives, rely on this kind of technology. This thesis serves as a guide to determine which preprocessing and feature configurations most improve outcomes for formal Japanese across different model types.

This study also contributes to the research by providing a systematic comparison of model choices in ASR across traditional, hybrid, and fine-tuned transformer models in the context of formal Japanese transcription. By filling this gap, the findings will inform best practices for ASR system design in Japanese, supporting both academic research and practical applications in industries that depend on accurate speech-to-text conversion.

1.7 Conclusion

In this chapter, the advancement in machine learning and artificial intelligence that has made the computer able to better understand humans' speech by improving the speech-to-text model accuracy and speed has been discussed. However, there are still challenges to transcribe a language that has a complex structure like Japanese, which includes syllable-based formation and the use of multiple writing systems. Because of this, a study to find which implementation and which model is the most performant for handling the Japanese language is needed. The findings from this study are important to answer the question of which model is the best speech-to-text solution in the Japanese language. By identifying the specific linguistic challenges and comparing these models, this study will provide valuable information that will guide future advancements in speech-to-text technology in the Japanese language and ultimately will support its broader application across industries that rely heavily on precise and efficient transcription.

CHAPTER TWO
LITERATURE REVIEW

2.1 Introduction

The technology for Automatic Speech Recognition (ASR) has advanced rapidly in recent years. Early traditional models like GMM and HMM have evolved into more sophisticated deep learning approaches such as DNN, CNN, RNN, and Transformer-based architectures.

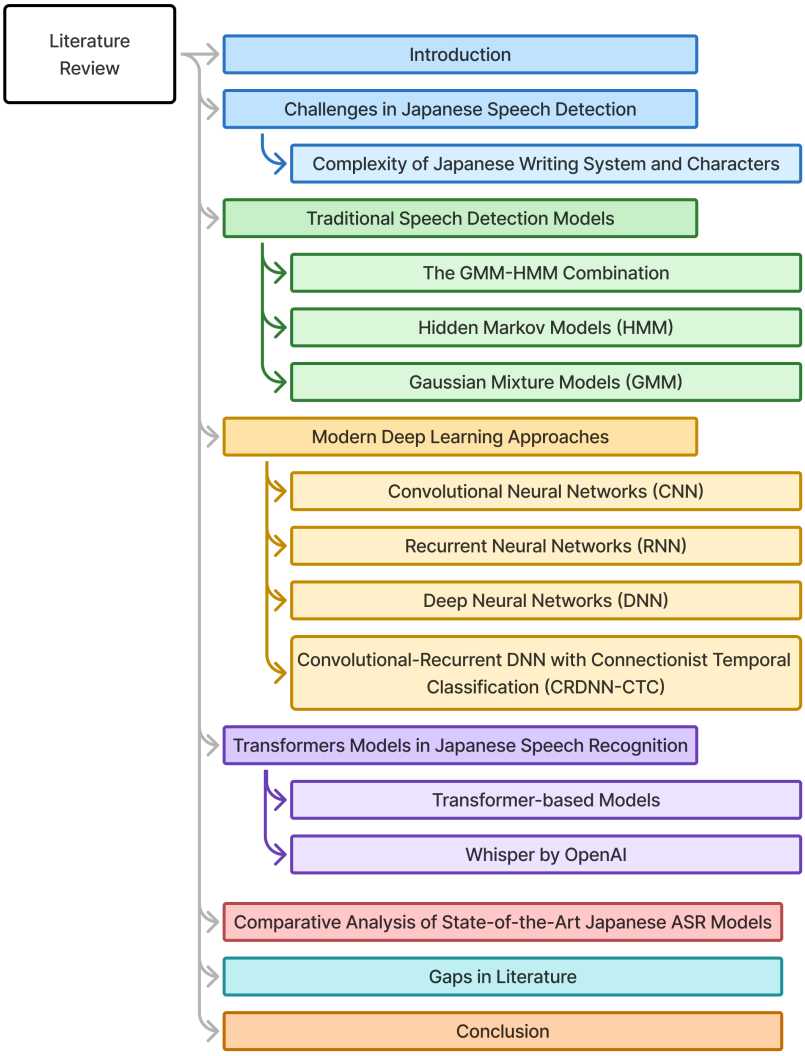

Figure 2.1 Literature Review Mind Map.

However, applying these technologies to the Japanese language may pose significant challenges due to its complex writing systems. In this chapter, the challenges

of Japanese speech recognition and the traditional and modern ASR models are reviewed. By identifying the key challenges and gaps in existing research, this chapter prepares for a focused analysis of Japanese-specific ASR systems.

2.2 Challenges in Japanese Speech Recognition

2.2.1 Complexity of Japanese Writing System and Characters

The complexity of the Japanese writing system and characters can cause challenges in ASR systems, especially in end-to-end neural network architectures. The Japanese writing system is a combination of multiple character sets, such as Hiragana, Katakana, Kanji (ideographic characters), Roman letters, and various symbols, leading to a considerably larger and more varied character set (Rose, 2019). As mentioned by Ito et al. (2016, 2017), the number of possible Japanese character labels can exceed several thousand.

A single kanji character may have multiple context-dependent pronunciations to pronounce it because each kanji character has Onyomi (Chinese derived) and Kunyomi (native Japanese) readings, and these readings can change depending on the word context (Curtin, 2020). Because of this ambiguity, the ASR system must be able to model and distinguish numerous acoustic differences in the speech data with the same sound. The training model must be able to handle thousands of characters, and each character is potentially linked to multiple context-dependent phonetic outcomes, which requires significant computational resources and large-scale training data to ensure adequate coverage (Ito et al., 2016, 2017).

2.3 Traditional speech recognition Models

2.3.1 Gaussian Mixture Models (GMM)

GMM has been one of the earliest technologies used for developing Japanese speech recognition and recognition systems because of its capability in capturing the statistical distribution of speech features very well (Imaishi & Kawabata, 2022). Because of the absence of word boundaries and the nuances of pitch accent in the Japanese language, it is highly complex to understand the context of the spoken words.

However, GMM is useful by using probabilities to manage and characterize intricate patterns (Sun & Chol, 2020). For example, Povey et al. (2011) were able to use GMM to model phoneme-based acoustic features, and this approach led to good performance in speech recognition systems.

Imaishi and Kawabata (2022) developed an approach within the EM algorithm that led to the stabilization of the GMM parameters as well as increasing the discriminative power of the model in cases where there is not much evaluation data available. In other work, Povey et al. (2011) pointed out that it is possible to represent the distribution of speech features in GMM mode by employing a combination of several Gaussian components. Takami and Kawabata (2020) emphasized a different direction which started with the creation of the Universal Background Model, which is a Gaussian Mixture Model calculated from the collection of a large number of speech samples.

2.3.2 Hidden Markov Models (HMM)

HMM works quite well with Japanese speech recognition because of the incorporation of the acoustic and temporal characteristics of speech, including the difficulties found in the encoding of Japanese speech (Tokuda, 1999). ASR systems incorporated with HMM are superior in portraying Japanese speech characteristics' rhythm and tone, including essential features like pitch accent and moraic timing, which enhance the performance of the systems on the phonology aspects of the language (Tokuda, 2000).

To further increase Japanese ASR capability, a few other models are used along with HMM which are context-sensitive, such as the triphone method. The triphone method is a phonetic expansion that employs phonetics of the neighbor sounds to the phoneme as context in order to increase the recognition accuracy by taking into account the co-articulation that takes place during fast speech production (Tokuda, 2000).

2.3.3 The GMM-HMM Combination

In the Kaldi framework, Japanese large-vocabulary ASR has commonly been implemented as a GMM–HMM pipeline where HMM states capture temporal dynam-

ics and GMMs model frame-level acoustic likelihoods, typically paired with context-dependent decision-tree clustered triphones. Classic Kaldi recipes also report step-wise gains when moving from simpler triphone systems to stronger feature-space modeling. For example, the Kaldi CSJ recipe reports that performance improves from early triphone stages to later stages, with eval1 WER decreasing from 22.67% (tri1) to 21.49% (tri2) and further to 17.49% (tri3) and 15.26% (tri4), demonstrating the typical benefit of successive refinements in the conventional HMM-based pipeline (Trmal, 2015).

More recently, Ando and Fujihara (2021) evaluated Kaldi-style HMM-based systems using the official CSJ recipe as a baseline training procedure and reported CER on both an out-of-domain TEDx-style test set and the standard CSJ evaluation sets. On the TEDxJP-10K evaluation set, their best reported configuration achieved 17.92% CER, while simpler configurations such as a CSJ acoustic model with a CSJ language model achieved 23.41% CER. These results provide concrete reference points for conventional HMM-based Japanese ASR performance under both matched (CSJ) and more realistic, diverse conditions (TEDxJP-10K) (Ando & Fujihara, 2021).

Table 2.1 ASR decoding results (CER%) on the TEDxJP-10K dataset adapted from Ando and Fujihara, 2021.

LM	**CSJ**	**TV**	**CSJ+TV**
CSJ	23.41	21.83	20.61
TV	22.14	18.98	18.22
OSCAR	23.52	19.28	18.71
TV+OSCAR	21.89	18.29	17.92

2.4 Modern Deep Learning Approaches

2.4.1 Deep Neural Networks (DNN)

The use of DNN in conjunction with HMM, also known as DNN-HMM, has been shown to improve performance in Japanese ASR. Seki et al. (2014) compared syllable-based and phoneme-based DNN-HMM and found that the syllable-based DNN-HMM was better, as its parameter space is less coupled with the context of the syllables. They reported an 11% relative decrease in the WER for triphone DNN-HMMs over syllable-based DNN-HMMs when used on large databases such as

ASJ+JNAS. The multilayered structure of DNNs makes it more suitable for developing models of contextual dependencies for speech signals (Hojo et al., 2018).

Mu et al. (2020) developed a double-deep neural network for the evaluation of Japanese pronunciation to address the problems of text-to-speech alignment and scoring. The DDNN integrated CNN and RNNs with attention and is effective for detecting pronunciation mistakes. Lin et al. (2017) noted the importance of addressing the particular problem of the lack of annotated Japanese speech corpora by emphasizing the use of transfer learning with DNN.

2.4.2 Convolutional Neural Networks (CNN)

Noda et al. (2014) conducted research using elastic net regression on a seven-layer CNN structure and obtained 58% phoneme recognition accuracy for Japanese datasets. Building upon this work, Yalta et al. (2019) constructed a functional speech recognition framework inclusive of several types of spoken words intended for enclosed acoustic environments such as domestic settings.

Noda et al. (2014) investigated the use of CNNs for solving the problem of creating a Japanese speech acoustic model. CNN is used to encode the frequency-time domain audio and properly exploit the spatial and temporal aspects. The combination of CNNs with attention mechanisms has been beneficial in increasing accuracy and interoperability during the detection of long utterances and multi-speaker datasets (Kohei Mukohara et al., 2015).

2.4.3 Recurrent Neural Networks (RNN)

In the work of Takeuchi et al. (2020), RNN is introduced, which enables the processing of input speech while removing noise and helps to reduce exploding gradient problems often seen in RNNs. Yusuke Kida et al. (2016) investigated linear prediction filters based on LSTM, which did not require direct access to raw information and thus can extract features from distorted signals, as an LSTM estimated linear prediction coefficients.

Kubo (2014) broadened approaches incorporating RNNs into synthesizing speech for Japanese, focusing on improving prosody and intonation. Takeuchi et al. (2020) took advantage of the RNN-based architectures for the acoustic modelling

for Japanese ASR and showed that even though GRUs have a simpler gating strategy than LSTMs, they could achieve a similar level of classification accuracy with lower computational requirements. Then, studies on bidirectional LSTMs (BLSTMs) by Imaizumi et al. (2022) revealed that the past context and the future context of the signal can be utilized for better performance of ASR.

2.4.4 Convolutional-Recurrent DNN with Connectionist Temporal Classification (CRDNN-CTC)

The CRDNN-CTC family is part of the broader end-to-end ASR direction that replaces the conventional lexicon-driven alignment pipeline with direct sequence learning. In this approach, acoustic features are mapped to output symbols using a neural encoder, and the Connectionist Temporal Classification (CTC) objective provides a monotonic alignment between the input frames and the target transcription without requiring frame-level labels. This has been used in Japanese ASR because Japanese can be transcribed naturally at the character or kana level, while word boundaries are not explicitly marked in continuous speech, making forced alignment and word-level tokenization less straightforward (Watanabe et al., 2017, 2018).

A consistent finding in prior work is that CTC-based models can achieve strong Japanese recognition accuracy while maintaining a simple training pipeline. For example, Chen et al. (2020) proposed an end-to-end streaming Japanese speech recognition method that combines CTC with local attention and reported a best CER of 9.87%, demonstrating that competitive character-level performance is possible even under streaming constraints where future context is limited. This line of work also supports the practicality of CTC-style training for Japanese, where stable alignments can be learned directly from paired audio-transcript data without the need of pronunciation dictionaries (Chen et al., 2020).

Table 2.2 Streaming Japanese ASR results using CTC and local attention from Chen et al., 2020.

Model no.	CNN	Attn.	Latency (ms)	Avg. CER (%)
1			40	12.60
2			60	13.67
3		✓	240	10.37
4		✓	360	10.40
5	✓		40	10.03
6	✓		60	12.27
7	✓	✓	240	9.87
8	✓	✓	360	10.27
9	✓	✓	360	10.23

2.5 Transformer Models in Japanese Speech Recognition

2.5.1 Transformer-based Models

Taniguchi et al. (2022) proposed a series of Transformer-based ASR models aimed at improving Japanese speech recognition, particularly in the context of simultaneous interpretation. They investigated the possibility of utilizing auxiliary input like the source language text to resolve issues such as disfluencies, hesitations, and self-repairs commonly observed in interpreter speech, which helps to improve the transcription quality (Futami et al., 2020). The models combined audio and text data via multi-modal transformer encoders and decoders, which offers a broader scope of recognition by using previously provided source language text for interpreter training programs (Taniguchi et al., 2022).

However, a wide range of datasets for source text and simultaneous interpretation speech are not readily available, so the authors used adapted speech translation corpora from MuST-C and CoVoST 2 while also introducing TED-based Japanese texts for evaluation purposes (Taniguchi et al., 2022). With an additional goal of enhancing performance, the authors fine-tuned the source language text encoder by using large machine translation corpora, which helps in lowering the word error rates during translation of English, Dutch, German, and Japanese (Taniguchi et al., 2024). Results consistently demonstrate that incorporating source language text into Transformer-based ASR models significantly improves recognition performance, with the greatest impact observed when auxiliary input is introduced at later stages of

the audio encoding and decoding process (Futami et al., 2020).

2.5.2 Whisper by OpenAI

Large-scale weak supervision has emerged as one of the major approaches in speech recognition, as noted by Radford et al. (2023) in their development of the Whisper model that has been trained on multilingual and multitask audio datasets that have a combined duration of 680,000 hours. This work is a continuation of self-supervised methods such as Wav2Vec 2.0 (Baevski et al., 2020), which demonstrated learning without supervision from audio without any human-provided labels. However, dataset-specific fine-tuning is often necessary to obtain good performance, whereas with Whisper such reliance is reduced because of the efficacy of weak supervision.

By scaling weak supervision across diverse datasets, Whisper is able to bypass the need for dataset-specific adaptation while being able to offer robust zero-shot performance across languages and tasks. This method also results in models having similar trends with other state-of-the-art models in machine learning, where large, diverse datasets improve model resilience, which aligns with advancements in computer vision (Kolesnikov et al., 2020) and NLP (Radford et al., 2019). The Whisper model's architecture, a simple encoder-decoder Transformer, reinforces the effectiveness of minimal preprocessing and sequence-to-sequence training, simplifying the transcription pipeline.

Bajo et al. (2024) detailed their work on adapting OpenAI's Whisper model to enhance its performance in ASR for the Japanese language. The research drew attention to the trade-off between multilingual capability and the accuracy of a Japanese-only product, ReazonSpeech, that seeks to maximize Japanese language ASR, which is monolingual in nature. By using a Japanese dataset while utilizing Low-Rank Adaptation (LoRA) and fine-tuning methods, they were able to lower Whisper-Tiny's CER from 32.7% to 14.7%. This fine-tuning method showed that smaller multilingual models give more promising results after being tuned for the desired language, outperforming their larger baseline models, for example in the case of the Whisper-Base model (Bajo et al., 2024).

Table 2.3 WER and CER performance of Whisper models. Reproduced from Bajo et al., 2024.

Model	**WER (%)**	**CER (%)**
Whisper Tiny	47.48	32.74
Whisper Base	29.81	20.20
Whisper Small	16.14	9.89
Whisper Medium	10.84	6.86
Whisper Large	7.41	4.77
Whisper Tiny + LoRA	33.16	20.83
Whisper Base + LoRA	23.36	14.50
Whisper Small + LoRA	14.90	9.16

2.6 Comparative Analysis of State-of-the-Art Japanese ASR Models

A comparative analysis carried out by Karita et al. (2021) shows that Conformer-based models perform better than Conformer BLSTM architectures, as they obtained 4.1, 3.2, and 3.5 character error rates for CSJ in eval1, eval2, and eval3 tasks respectively. It is noted that both the BLSTM and Conformer models have character error rates below 7% and the character error rate is lower when using Conformer itself. Conformer encoders also offer increased accuracy and efficiency, with a throughput of 628.4 utterances processed per second and 430.0 for the BLSTM models (Karita et al., 2021).

Table 2.4 Character error rates on CSJ dev/eval1/eval2/eval3 sets cited from Karita et al., 2021.

Encoder	**Decoder**	**Param**	**Utt/sec**	**CER [%]**
BLSTM	CTC	258M	430.0	3.9 / 5.2 / 3.7 / 4.0
BLSTM	attention	309M	365.5	3.8 / 5.3 / 3.7 / 3.7
BLSTM	transducer	274M	297.6	3.8 / 5.1 / 3.7 / 4.0
Conformer	CTC	117M	**628.4**	3.1 / 4.1 / 3.2 / 3.5
Conformer	attention	124M	534.8	3.3 / 4.5 / 3.3 / 3.5
Conformer	transducer	120M	376.1	**3.1 / 4.1 / 3.2 / 3.5**

Another comparative analysis in ASR for Japanese is presented by Takahashi et al. (2024), focusing on the accuracy of speech recognition for different dialects. The study evaluated three models: Whisper, XLSR, and XLS-R, which are self-supervised learning frameworks. The Whisper model significantly underperformed for any Japanese outside of standard Japanese, recording a 4.1% CER only after it had gone through fine-tuning. However, the accuracy is low when the language identification marker is absent, with some instances being higher than 100%. This marks

the weakness of Whisper in terms of its application for wide-ranging applications in different dialects of Japanese (Takahashi et al., 2024).

Table 2.5 Comparison of ASR accuracy on two datasets, Standard Japanese (CSJ) and Japanese dialects (COJADS) cited from Takahashi et al., 2024.

Pre-Trained Model Name	Adaptation Method	CER [%] CSJ	CER [%] COJADS
Whisper-medium (zeroshot)	-	25.6*	116.0*
Whisper-medium	full finetuning	**4.1**	32.9
XLSR	full finetuning	6.5	34.1
XLSR	3-steps finetuning	-	30.0
XLS-R	full finetuning	6.1	32.6
XLS-R	3-steps finetuning	-	**29.2**

*After post-processing with kanji-to-kana conversion.

A recent comparative study by Hono et al. (2024) proposed an end-to-end Japanese ASR framework that integrates a pre-trained speech encoder (HuBERT) with a decoder-only large language model (GPT-NeoX) via a bridge network, aiming to avoid explicit LM-fusion and keep decoding simple. The evaluation compared the proposed model against publicly available baselines across several Japanese corpora. For efficiency, the authors reported that applying DeepSpeed-Inference improved inference speed by about 1.4× while keeping recognition accuracy unchanged. In terms of CER, the proposed model outperformed reazonspeech-espnet-v1 across all beam settings on the ReazonSpeech test set (in-domain), while showing mixed outcomes on JSUT and CV8.0 compared to larger-beam decoding.

Table 2.6 ASR results reported by Hono et al., 2024 on ReazonSpeech, JSUT, CV8.0, and CSJ eval sets (beam size 1 = greedy decoding).

Model	Params	Beam	Reazon	JSUT	CV8.0	CSJ E3
reazonspeech-espnet-v1	90M	1	12.0	8.7	10.5	15.3
reazonspeech-espnet-v1	90M	20	8.7	7.5	8.9	15.5
Whisper-large-v3	1,541M	1	12.4	7.1	8.2	14.8
Proposed (w/o DS-Inf.)	3,708M	1	8.4	8.6	9.1	22.9
Proposed (w/ DS-Inf.)	3,708M	1	8.4	8.6	9.2	22.9

Current comparative analysis from these studies shows that several challenges need to be addressed. A study to compare existing models in Japanese ASR is needed because of the unique characteristics of the Japanese language. The comparative analysis from Karita et al. (2021) and Takahashi et al. (2024) shows that while models like Conformer and Whisper have made significant strides in ASR, they still face chal-

lenges in handling the complexities of Japanese. Additionally, there is still a lack of studies that compare models from different model families because most existing comparative analyses only focus on models from the same family.

2.7 Gaps in Literature

Based on the literature review, there are several gaps in the research on Japanese ASR. First, while there has been significant progress in ASR models, there is still a lack of comprehensive comparative studies that evaluate different model architectures specifically for Japanese. Most existing studies focus on individual models or specific families of models, making it difficult to draw broad conclusions about the best approaches for Japanese ASR. Second, many studies use datasets that have already been thoroughly studied, such as CSJ, which has been cleaned and preprocessed extensively. There is a need for more research using diverse and real-world datasets, such as TEDx talks, to better understand model performance in practical scenarios.

2.8 Conclusion

This chapter has reviewed the existing literature on Japanese ASR, covering traditional models like GMM–HMM and modern deep learning approaches including CRDNN-CTC, and Transformer-based models like Whisper. The unique challenges posed by the Japanese language, such as its complex writing system and character pronunciations, were discussed. Tools and datasets commonly used in Japanese ASR research were also highlighted, and gaps in the literature were also identified.

CHAPTER THREE
RESEARCH METHODOLOGY

3.1 Introduction

This chapter explains the methodology used in this project to compare Japanese automatic speech recognition (ASR) across three model families. The models are a traditional Kaldi-style GMM–HMM system, an end-to-end CRDNN–CTC model, and a fine-tuned transformer encoder–decoder model based on the Whisper model from OpenAI (Ghoshal et al., 2011; Radford et al., 2023; Ravanelli et al., 2024). In this chapter, the data pipeline such as data collection, audio and transcript cleaning, and audio splits is described. The preprocessing, training, decoding process, and the scoring protocol used for each model family are also discussed in this chapter. Figure 3.1 displays the simplified workflow used in this study.

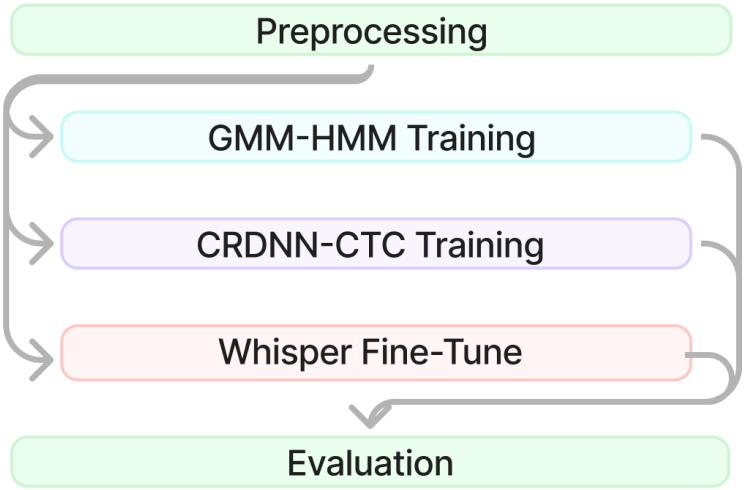

Figure 3.1 Simplified research methodology flow.

3.2 Tools Used

3.2.1 Python

Python was used in this project as the main programming language to implement the data pipeline, training utilities, and evaluation scripts. It supported preprocessing tasks such as transcript cleaning, manifest generation, and training split prepa-

ration (Python, 2025). In addition, Python libraries were used to support ASR-related workflows such as audio processing, metadata handling, diarization-based filtering, and evaluation.

3.2.2 Preprocessing tools

Yt-dlp is a command-line tool and Python library that allows downloading videos, audio, and subtitle tracks from YouTube. In this study, yt-dlp was used to query YouTube metadata to verify the availability of manual Japanese subtitles before downloading. Then, audio and subtitle files were downloaded for selected Japanese TEDx talks to form a consistent dataset for ASR training and evaluation (Rawat et al., 2023).

To standardize the dataset and ensure compatibility across model families, audio was converted into a consistent format which is 16 kHz, mono, and 16-bit PCM. Audio resampling and waveform writing were performed using Python audio libraries `librosa` and `soundfile` (McFee et al., 2025; PySoundFile, 2015). In addition, WebRTC voice activity detection was used to segment long TEDx recordings into shorter utterances suitable for training and evaluation (Blum et al., 2021).

A filtering step was applied to remove non-speaker or low-quality segments such as applause and laughter. Speaker structure filtering used `pyannote` diarization to prefer clips with a dominant speaker. Acoustic event filtering used a Hugging Face zero-shot audio classification pipeline with the CLAP model (`laion/clap-htsat-fused`) to identify and remove segments with high confidence of applause or laughter (Wu et al., 2022).

3.2.3 Traditional Models Training Tools

Kaldi is an open-source toolkit for speech recognition that provides a complete pipeline for feature extraction, acoustic modeling, and decoding (Ghoshal et al., 2011). In this study, Kaldi was used to implement the traditional GMM–HMM baseline system. The workflow included MFCC feature extraction with CMVN normalization, staged acoustic model training (mono, tri1, tri2, tri3), alignment between stages, and decoding graph generation for evaluation.

In this project, `lmplz` from KenLM was used to produce an ARPA-format language model, which was then converted into decoding resources within the Kaldi graph construction pipeline (Heafield, 2011). A grapheme-based lexicon was generated using a custom Python script. The script extracted vocabulary items from transcripts and converted Japanese tokens into Hepburn romanisation using `pykakasi` (Pykakasi, 2022).

3.2.4 Neural Models Training Tools

SpeechBrain is an open-source toolkit for speech processing that supports end-to-end ASR training and evaluation (Ravanelli et al., 2021). In this study, SpeechBrain was used to implement the CRDNN–CTC model. It used the prepared CSV manifests and a character-level vocabulary, and training checkpoints were managed using SpeechBrain's checkpointer mechanism (Ravanelli et al., 2024).

3.2.5 Whisper Fine-tuning Tools

Hugging Face was used to access pretrained Whisper models and supporting utilities for fine-tuning. The Transformers library was used to load Whisper checkpoints, configure Japanese decoding prompts, and generate hypotheses using `model.generate()`. Hugging Face Datasets was used to load train/validation/test CSV splits efficiently and support reproducible fine-tuning experiments (Wolf et al., 2020).

3.2.6 Evaluation Tools

Model evaluation used character error rate (CER), word error rate (WER), and real-time factor (RTF). CER and WER were computed using normalized edit distance between hypothesis and reference transcripts. In this study, the `jiwer` Python library was used to compute WER and to support consistent error-rate scoring across systems. RTF was computed by dividing total decoding wall time by total audio duration to measure decoding efficiency and latency behaviour (Jiwer, 2025).

3.3 Research Design

This study was organised into four phases where it started from a preliminary study. It then moved to the data collection and preprocessing phase where the dataset was compiled and processed. After that, the selected models were trained and fine-tuned using the data prepared from the earlier phase. Then, the models were scored using selected metrics and the output data were finally analyzed and discussed in the result and discussion phase. Table 3.1 below shows an overview of the phases, activities, and deliverables.

Table 3.1 Research Design

Phase	Activities	Deliverables
Preliminary Study	Defined below items: • Project title • Problem statements • Objectives • Scopes • Significance • Reviewed Japanese ASR studies • Listed preprocessing methods • Listed model families • Listed evaluation metrics • Summarized key gaps	Objective 1 • Title defined • Problems defined • Objectives defined • Scope defined • Significance defined • Literature review written • Preprocessing chosen • Models selected • Metrics defined • Gaps summarized
Data Collection and preprocessing	• Collected Japanese TEDx talks • Downloaded audio and subtitles • Cleaned and normalized transcripts • Resampled audio to 16 kHz mono • Split audio using VAD • Created train/valid/test splits • Exported manifests and metadata	Objective 2 • Dataset compiled • Collection method documented • Transcripts normalized • Segments generated • Splits finalized • Manifests prepared
Model Training and Fine Tune	• Trained GMM-HMM • Trained CRDNN-CTC • Fine-tuned Whisper variants • Set decoding parameters • Saved configs and checkpoints	Objective 2 • Models trained • Checkpoints saved • Settings recorded • Outputs generated
Result and Discussion	• Scored with CER and WER • Measured speed using RTF • Compared models and decoders • Explained main findings • Linked to prior work • Noted limitations • Suggested future work	Objective 3 • Results tables and plots • Best model identified • Limitations stated • Recommendations given • Future work proposed

3.4 Preliminary Study Phase

Table 3.2 shows the first phase of this project where most of the planning and decisions about this project were defined. This phase also reviewed the preliminary study and some decisions were made based on previous studies. The activities carried out and the outcome deliverables are described in the table shown below.

Table 3.2 Preliminary Study Phase

Phase	**Activities**	**Deliverables**
Planning	Defined below items: • Project title • Problem statements • Objectives • Scopes • Significance • Reviewed Japanese ASR studies • Listed preprocessing methods • Listed model families • Listed evaluation metrics • Summarized key gaps	Objective 1 • Title defined • Problems defined • Objectives defined • Scope defined • Significance defined • Literature review written • Preprocessing chosen • Models selected • Metrics defined • Gaps summarized

The preliminary study phase was important because it served as the blueprint and became guidance for the project direction. The purpose of this phase was to identify the gap in the domain and create a plan on how to solve the problem stated. The activities that were carried out in this phase were defining the title of the project, problem statements, objectives, scope, and significance of the project. The outcome from this phase was the title of the project, problems that had been defined, objectives of the project, and scope of the project which explained what was covered and what was not covered in this project. During this preliminary study phase, things like time, cost, resources, benefits, and difficulty level were also taken into consideration.

Another activity conducted during the preliminary study was reviewing multiple sources such as related articles, journals, conference papers, and textbooks. These resources were used to evaluate and identify what had been discussed or discovered. The activity involved reviewing the preprocessing methods that were currently being used from previous studies. The next activity was to choose which models to compare, instead of comparing models from the same family, this research decided to compare three models from different model architectures and these three selected models were the deliverables for this phase. The next activity was to conclude which metrics were used to compare these models. As the deliverables, three metrics were chosen to do comparative analysis between each model. In this phase, the gaps in the literature were also defined and as the deliverables the gaps in the literature were summarized.

3.5 Data Collection and Preprocessing Phase

Table 3.3 shows the second phase of this project where the source of the data was selected, downloaded, and then the data went through preprocessing to make sure the data was ready to be input into models. The activities carried out and the outcome deliverables are described in the table shown below.

Table 3.3 Data Collection and Preprocessing Phase

Phase	**Activities**	**Deliverables**
Data Collection and preprocessing	• Collected Japanese TEDx talks • Downloaded audio and subtitles • Cleaned and normalized transcripts • Resampled audio to 16 kHz mono • Split audio using VAD • Created train/valid/test splits • Exported manifests and metadata	Chapter 3 • Dataset compiled • Collection method documented • Transcripts normalized • Segments generated • Splits finalized • Manifests prepared

The focus of this phase was to build a cleaned and usable dataset from publicly available data into a consistent training and evaluation format so that all three models from different families could use the same dataset for training and testing (Ando & Fujihara, 2021). The pipeline for this phase could be separated into seven tasks. The first task was collecting Japanese TEDx talks from YouTube with Japanese subtitles that came with manual subtitles by humans. Then the audio was downloaded with the subtitle files. The second task was to clean and normalize subtitle text into reference transcripts. The third task was to resample audio into 16 kHz mono PCM. The fourth and fifth tasks were to split the audio into chunks using voice activity detection (VAD), and audio that did not have audio activity such as intro music was discarded. The last two tasks were to export manifests and metadata and then produce train/validation/test splits based on manifests and metadata with audio paths, durations, and transcripts. Figure 3.2 below shows the data collection and preprocessing pipeline used in this phase.

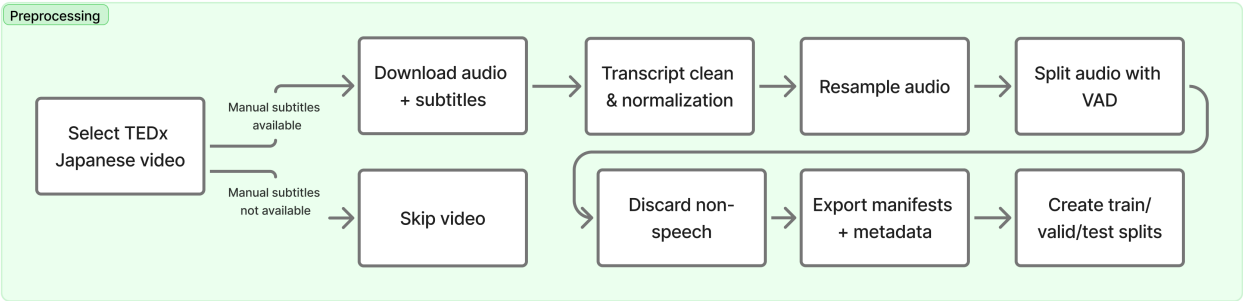

Figure 3.2 Data collection and preprocessing pipeline

3.5.1 Data Source and Selection Criteria

The source of the data was collected from Japanese TEDx talks that were hosted on the video sharing platform, YouTube. This particular source was chosen because TEDx talks were a collection of public speaking where commonly the speaker had clear speech with structured monologue-style delivery (Ando & Fujihara, 2021). However, not all videos that were published came with manual Japanese subtitles by humans, only part of the videos had manual Japanese subtitles. The rest either did not have subtitles or the subtitles were in another language. To make sure that the transcripts were sufficiently reliable to serve as reference text during ASR training and evaluation, only videos that had Japanese manual subtitles were chosen and the rest were excluded. Videos that contained only auto-generated subtitles were also excluded to reduce transcription noise and alignment inconsistencies.

All audio and subtitles were downloaded using a Python library named yt-dlp (Rawat et al., 2023). The script first queried the video metadata to find if the video in the playlist had Japanese subtitles or not without downloading the video using the `extract_info` function from the yt-dlp library (Rawat et al., 2023).

Listing 3.1: Query video metadata script

```
with yt_dlp.YoutubeDL(check_opts) as ydl:
    info = ydl.extract_info(video_url, download=False)
    if 'subtitles' in info and 'ja' in info['subtitles']:
        has_jp_subs = True
```

For videos that had Japanese subtitles, audio and transcripts were downloaded (Rawat et al., 2023).

Listing 3.2: Audio downloading script

```
download_opts = {'langs': ['ja'], 'format': 'vtt'}
if has_jp_subs:
    with yt_dlp.YoutubeDL(download_opts) as ydl:
        ydl.download([video_url])
```

3.5.2 Transcript Cleaning and Normalization

After the transcript was downloaded, the transcript needed to be cleaned since it could not be directly used as a reference transcript (Ando & Fujihara, 2021). The raw downloaded transcript contained non-speech markers, formatting tags, and artifacts introduced for readability rather than ASR training. All subtitle files were normalized into cleaned text using a custom script. The cleaning rules implemented included:

- Removing HTML and formatting tags such as `<i>`.
- Removing speaker labels such as `Speaker 1:`, `NARRATOR:`.
- Removing non-speech annotation events such as `[laughter]`, `(applause)`.
- Normalizing whitespace and punctuation.

This produced a transcript file where one cleaned line per subtitle cue was stored with the timestamp, which was useful for later alignment between subtitle cues and speech segments. The timestamp was used to separate the long audio format into chunks.

3.5.3 Audio Standardization to 16 kHz Mono

The downloaded audio may come with different formats based on the download option and video quality. To ensure compatibility with all selected models, the audio was standardized into 16 kHz mono. This was also to make sure fairness across all ASR model families when compared to each other. The chosen standard was a 16 kHz sample rate, mono channel, and 16-bit PCM because this format was a standard configuration used in Kaldi recipes and also aligned with typical front-end assumptions in many end-to-end ASR pipelines (Ghoshal et al., 2011).

Listing 3.3: Audio resampling to 16 kHz

```
def resample_16khz(in_wav, out_wav, target_sr=16000):
    y, sr = librosa.load(in_wav, sr=None)
    if sr != target_sr:
        y = librosa.resample(y, orig_sr=sr, target_sr=target_sr)
        sr = target_sr
    sf.write(out_wav, y, sr)
```

3.5.4 Speech Segmentation using WebRTC VAD

The downloaded audio was averaged between 20 minutes to 30 minutes. This duration was not suitable to be used as training data because it was inefficient for the model to learn effectively from very long utterances, and it also increased memory usage and training time. To deal with this issue, WebRTC voice activity detection (VAD) was used to split long audio files into chunked audio suitable to be used in the training phase (Blum et al., 2021). The segmentation strategy was:

- Audio was split into 30 ms frames.
- Speech frames were grouped into continuous speech regions.
- A region was closed when silence exceeded a threshold of 500 ms.
- Very short segments less than 2 seconds were removed.
- Very long segments more than 20 seconds were further sliced into chunks.

This process resulted in numerous short audio files and to manage all the chunked files, the naming convention below was used to make sure the original audio name was still available:

`<originalname>_chunk_000.wav`, `<originalname>_chunk_001.wav`, ...

This naming convention was later reused as utterance IDs in the transcript mapping and manifests.

3.5.5 Removal of Non-Speech Audio

To improve the quality of the dataset, a filtering step was applied to remove audio that did not have any Japanese speakers in it. This was because this audio be-

came noise and could cause the quality of the training data to become lower. The type of audio that was removed was audio clips that were dominated by applause/laughter and heavy background noise (Wu et al., 2022). A Python script was created to filter out this type of audio by applying two checks:

- Speaker structure check: clips were preferred when a single dominant speaker was present.
- Acoustic event detection: clips with high confidence of laughter or clapping were removed based on threshold scores.

Listing 3.4: Automatic audio quality filtering

```
for audio_path in wav_files:
    dom_ratio = dominant_speaker_ratio(diar(audio_path))
    scores = clap(audio_path,
        candidate_labels=["people laughing", "applause or hand clap"])
    clear = ((dom_ratio >= 0.85) and (score(scores, "laugh") < 0.30)
        and (score(scores, "clap") < 0.30))

    if not clear:
        os.remove(audio_path)
```

Files failing the clarity or language criteria were deleted automatically, leaving a cleaner set of speech segments for the downstream ASR experiments.

3.5.6 Transcript-to-Audio Mapping and Manifest Preparation

The next step after splitting the audio into chunks and cleaning the subtitles was to align the audio with a matching reference transcript to enable supervised training and evaluation. The cleaned subtitle text served as the transcript source and the final transcript mapping was stored in a simple format like this:

`utt_id: transcript`

where `utt_id` matched the chunk filename and the `transcript` was the cleaned subtitle text that had been extracted and aligned based on the timestamp in the subtitle

file. After the reference transcript was aligned with the audio files, for training and evaluation reproducibility, metadata manifests were generated containing:

- `utt_id` (utterance identifier),
- `wav` (absolute path to the audio file),
- `duration` (in seconds),
- `transcript` (cleaned reference text).

3.5.7 Train/Validation/Test Splits

The final dataset was divided into three splits to support model development and unbiased evaluation. The split ratio used was 80% training, 10% validation, and 10% testing. The data was split like this because the training set must be large enough for the models to learn the language patterns, while the validation set was required to tune hyperparameters and select checkpoints without leaking information from the test set. The test set was kept completely unseen until the final evaluation to provide an unbiased estimate of real-world transcription performance.

3.6 Model Training and Fine-Tuning Phase

Table 3.4 shows the fourth phase of this project where cleaned audio and transcripts earlier were used to train or fine-tune models. The main objective of this phase was to make sure each model was trained using the same data split so that the evaluation in the next phase was fair and consistent.

Table 3.4 Model Training and Fine-Tuning Phase

Phase	Activities	Deliverables
Model Training and Fine Tune	• Trained GMM-HMM • Trained CRDNN-CTC • Fine-tuned Whisper variants • Set decoding parameters • Saved configs and checkpoints	Chapter 4 • Models trained • Checkpoints saved • Settings recorded • Outputs generated

The models used in this phase were a traditional Kaldi GMM–HMM, an end-to-end CRDNN–CTC using the SpeechBrain framework that was trained from zero, while a transformer encoder–decoder Whisper model from OpenAI was fine-tuned us-

ing the same dataset. The diagram in Figure 3.3 below shows the simplified workflow for training and fine-tuning the models used in this project.

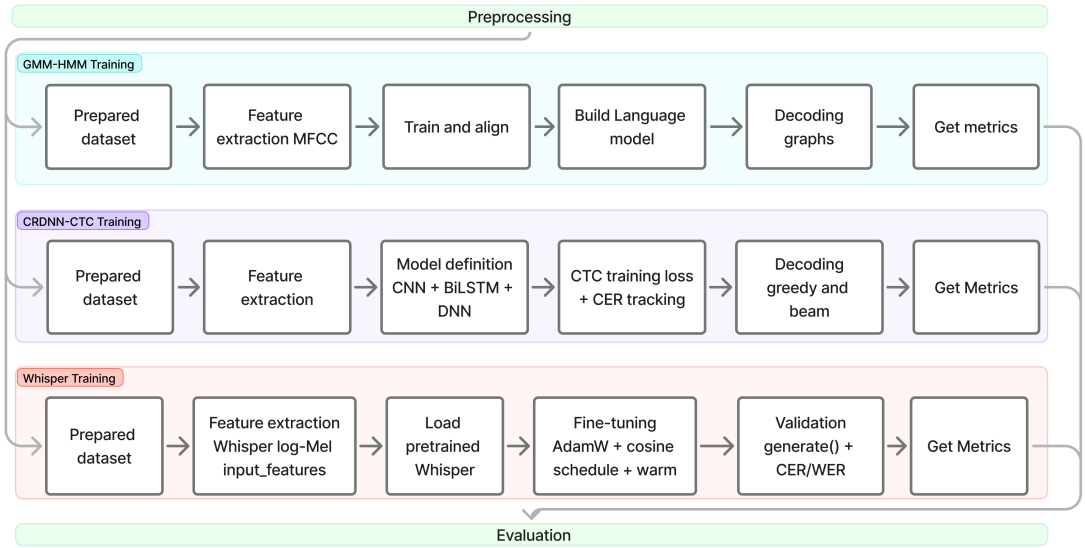

Figure 3.3 Simplified model training and fine-tuning workflow.

3.6.1 GMM–HMM Training (Kaldi)

The first model family was the traditional Kaldi-style GMM–HMM system. This model was trained using a standard Kaldi recipe flow which started from feature extraction, followed by acoustic model training in multiple stages (mono, tri1, tri2, tri3). In this experiment, MFCC features with CMVN normalization were used because it was a common baseline setup in Kaldi and suitable for classical HMM-based modelling (Ghoshal et al., 2011).

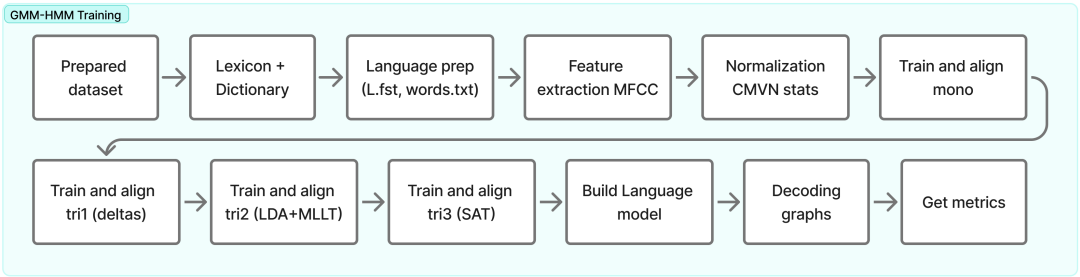

Figure 3.4 Model training workflow for GMM-HMM.

3.6.1.1 Lexicon, Dictionary, and Language Preparation

A pronunciation dictionary and lexicon were prepared using a Python script. The script extracted a vocabulary list from the training transcripts and generated a

grapheme-based lexicon by converting each Japanese token into Hepburn romanisation using `pykakasi` (Pykakasi, 2022). The romanised output was normalised to lowercase and filtered to `a-z` characters only. Each word was then mapped to a sequence of letters, which served as the basic pronunciation units (phones).

Listing 3.5: Grapheme-based lexicon generation

```
for w in vocab:  # unique tokens from training transcripts
    roma = "".join(x["hepburn"] for x in conv.convert(w)).lower()
    phones = list(re.sub(r"[^a-z]", "", roma))
    if phones:
        lexicon.write(f"{w} {' '.join(phones)}\n")
        phone_set.update(phones)
```

Next, an unknown token `<unk>` was inserted into the lexicon to handle out-of-vocabulary (OOV) terms during decoding. This was for silence between speech or unknown features when decoding. After that, a language directory `data/lang` which contained symbol tables and language resources, was created using Kaldi internal recipe. The output of this step was a complete language directory that included `words.txt`, `L.fst`, and other required files used by Kaldi for graph construction.

3.6.1.2 Feature Extraction (MFCC + CMVN)

MFCC (Mel-Frequency Cepstral Coefficients) are compact numeric features that were extracted from audio to represent the way humans hear sound. It converted audio into numerical values and was used as input features in the ASR system. MFCC features were extracted for train, dev, and test sets using `steps/make_mfcc.sh` from Kaldi recipe.

After that, CMVN (Cepstral Mean and Variance Normalization) statistics were computed using `steps/compute_cmvn_stats.sh`. CMVN was a normalization step that was applied to the MFCC to make the features more consistent by removing channel/mic/loudness differences. It also applied variance normalization to scale the unit variance and reduced scale differences. Listing 3.6 below is the script to run MFCC and CMVN:

Listing 3.6: MFCC and CMVN feature extraction

```
for s in train dev test; do
  steps/make_mfcc.sh --mfcc-config conf/mfcc.conf --nj 4 \
     --cmd run.pl data/$s exp/make_mfcc/$s mfcc
  steps/compute_cmvn_stats.sh data/$s exp/make_mfcc/$s mfcc
done
```

3.6.1.3 Acoustic Model Training Stages

To train the acoustic model, training was done gradually to improve the performance (Ghoshal et al., 2011). The acoustic model was trained using four stages as described below:

- Monophone (mono): a baseline context-independent model trained using `train_mono.sh`.
- Triphone 1 (tri1): a delta-based triphone model trained using `train_deltas.sh`.
- Triphone 2 (tri2): a triphone model with LDA+MLLT transforms trained using `train_lda_mllt.sh`.
- Triphone 3 (tri3): a speaker-adaptive training (SAT) model trained using `train_sat.sh`.

Between each stage, the training set was aligned using `align_si.sh` to produce improved alignments for the next model. This staged approach was important in Kaldi because better alignments usually led to better acoustic models (Ghoshal et al., 2011). Listing 3.7 below was the snippet of the bash script used to run the training recipe from the Kaldi library.

Listing 3.7: Acoustic model training stages

```
steps/train_mono.sh --nj 1 --cmd run.pl \
  data/train data/lang exp/mono
steps/align_si.sh --nj 1 --cmd run.pl \
  data/train data/lang exp/mono exp/mono_ali
steps/train_deltas.sh 2000 10000 \
  data/train data/lang exp/mono_ali exp/tri1
steps/align_si.sh --nj 1 --cmd run.pl \
  data/train data/lang exp/tri1 exp/tri1_ali
```

```
steps/train_lda_mllt.sh 2500 15000 \
  data/train data/lang exp/tri1_ali exp/tri2
steps/align_si.sh --nj 1 --cmd run.pl \
  data/train data/lang exp/tri2 exp/tri2_ali
steps/train_sat.sh 3500 20000 \
  data/train data/lang exp/tri2_ali exp/tri3
```

3.6.1.4 Language Model Construction and Decoding Graph

For decoding using a 3-gram, a language model was created using KenLM (Heafield, 2011). An external language model from a Wikipedia dump was used because it had a huge number of vocabulary to cover a huge amount of language patterns. After that, `lmplz` was used to produce an ARPA LM, and Kaldi tools were used to convert the ARPA file into `G.fst` (Ghoshal et al., 2011; Heafield, 2011).

Finally, decoding graphs were created for each acoustic model using `utils/mkgraph.sh`. This produced graphs such as `exp/mono/graph`, `exp/tri1/graph`, and so on, which were used for decoding the test set (Ghoshal et al., 2011).

Listing 3.8: Decoding graph creation

```
utils/mkgraph.sh data/lang exp/mono exp/mono/graph
utils/mkgraph.sh data/lang exp/tri1 exp/tri1/graph
utils/mkgraph.sh data/lang exp/tri2 exp/tri2/graph
utils/mkgraph.sh data/lang exp/tri3 exp/tri3/graph
```

3.6.2 CRDNN–CTC Training (SpeechBrain)

The second model family was from neural network models and the CRDNN-CTC model was chosen based on the literature review. This model was trained using the SpeechBrain framework with the goal of learning the direct mapping from acoustic features to output character sequences (Ravanelli et al., 2024). For the decoder setup, CTC was used because Japanese transcripts could be handled naturally as a sequence of characters, and it avoided the need for word segmentation (Chen et al., 2020).

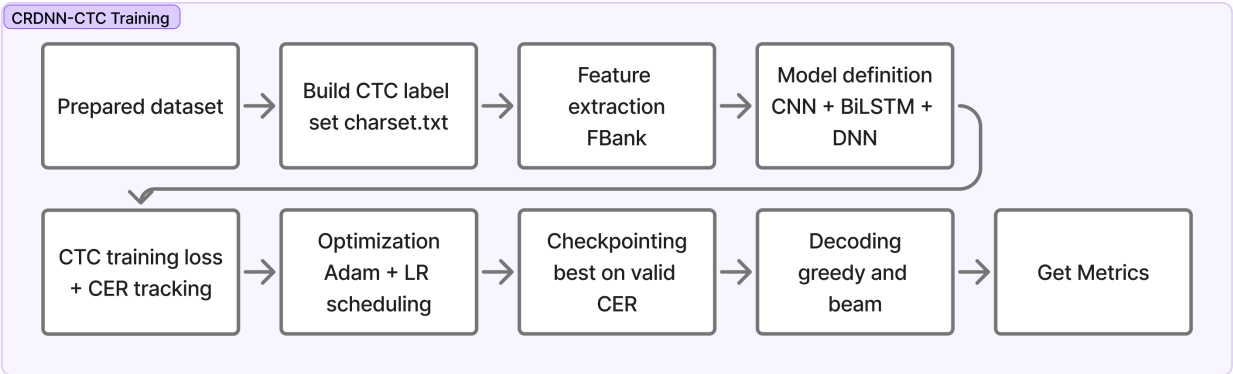

Figure 3.5 Model training workflow for CRDNN-CTC.

3.6.2.1 Manifest Preparation

Similar to the GMM-HMM setup earlier, the CRDNN-CTC setup also utilized the same CSV manifests that included `ID`, `wav`, `duration`, and `transcript`. The manifests were generated from the prepared wav directory and transcript mapping file. The same 80/10/10 split strategy train/valid/test was used.

3.6.2.2 Character Vocabulary (CTC Token Set)

The next step was to create a character vocabulary from the training script by scanning all characters in the training set and writing them into a charset file. The vocabulary file began with a special `<blank>` token to represent the CTC blank symbol. After that, this charset was loaded into SpeechBrain using `CTCTextEncoder` (Ravanelli et al., 2024). This step was important because CTC required a fixed label set, and the model output layer size depended directly on the number of characters in this vocabulary (Graves & Jaitly, 2014). By generating the charset from the training transcripts, the label set stayed consistent with the dataset and reduced unexpected errors when unseen symbols appeared during training.

Listing 3.9: Character vocabulary generation for CTC

```
df = pd.read_csv("train.csv")
chars = Counter(ch for t in df.transcript.astype(str)
                for ch in ud.normalize("NFKC", t).strip() if ch != " ")
with open("charset.txt", "w") as f:
```

```
    f.write("<blank>\n" + "\n".join(sorted(k for k in chars)) + "\n")
```

3.6.2.3 Model Configuration and Hyperparameters

The CRDNN model hyperparameters were defined in `hyperparams.yaml`. In this experiment, filterbank (FBank) features with mean-variance normalization were used. The architecture consisted of CNN blocks, followed by bidirectional LSTM layers, and a DNN block before a final linear layer that projected to the CTC vocabulary size. The optimizer used was Adam with a learning rate of 1×10^{-3}, and learning rate scheduling was handled using NewBob scheduling based on validation improvement (Ravanelli et al., 2024). Dropout was also applied to reduce overfitting because the dataset size was limited compared to large-scale ASR corpora. In addition, batch sizes for training and validation were configured separately to make sure validation could run stably under limited GPU memory.

3.6.2.4 Training Procedure and Checkpointing

A custom SpeechBrain `Brain` class (`ASRBrain`) was implemented to define forward computation, CTC loss, and CER tracking during validation. During training, checkpoints were saved using the SpeechBrain checkpointer so that the best model could be recovered and used during evaluation. The model was trained for a fixed number of epochs, and validation CER was monitored across epochs. The learning rate was automatically adjusted when validation improvement became small, which helped stabilize training and avoided over-updating the model. At the end of training, the best checkpoint was selected based on validation performance (Ravanelli et al., 2024).

3.6.3 Whisper Fine-Tuning (Transformer Encoder–Decoder)

The third model family was Whisper, a transformer encoder–decoder ASR model from OpenAI. In this research, multiple Whisper variants were fine-tuned including tiny, base, small, medium, and large-turbo. Whisper was fine-tuned using supervised learning by providing audio features as inputs and reference transcripts as labels (Radford et al., 2023).

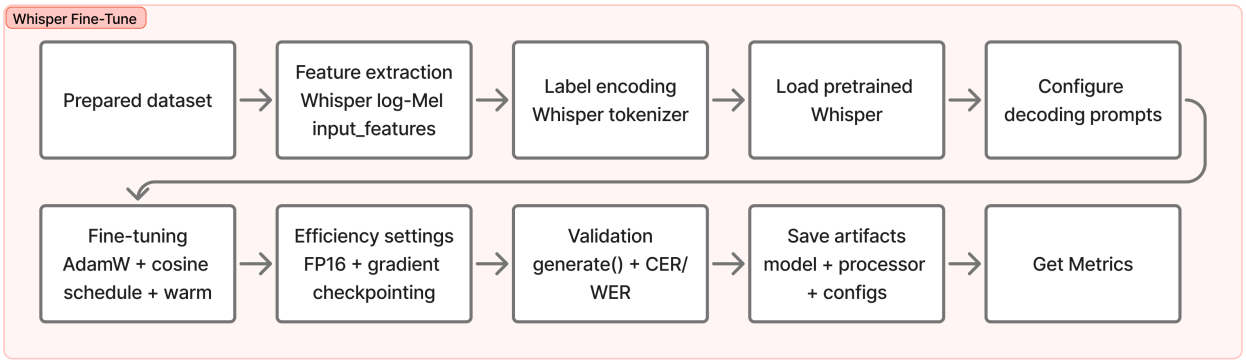

Figure 3.6 Model training workflow for Whisper.

3.6.3.1 Dataset Preparation for Fine-Tuning

The dataset was prepared by converting `transcript_utf8.txt` into a DataFrame that mapped each utterance ID to its corresponding `.wav` path and transcript. The data was shuffled using a fixed random seed for reproducibility and split into training, validation, and testing sets. Each split was stored as CSV files (`train.csv`, `valid.csv`, `test.csv`) and later loaded using HuggingFace `datasets` (Wolf et al., 2020).

3.6.3.2 Feature Extraction and Label Encoding

Whisper used log-Mel filterbank features extracted by the Whisper feature extractor. In preprocessing, each audio sample was resampled to 16 kHz and converted into `input_features`. The transcript text was tokenized using the Whisper tokenizer to generate `labels` (Radford et al., 2023).

3.6.3.3 Fine-Tuning Configuration

For fine-tuning, a pretrained Whisper checkpoint was loaded. The decoder was configured with Japanese language prompts using forced decoder IDs (Wolf et al., 2020). Gradient checkpointing-related settings were applied by setting `use_cache=False` (Wolf et al., 2020). Training used the AdamW optimizer with a small learning rate and cosine learning rate scheduling with warmup. Mixed precision training was enabled when running on GPU to speed up training and reduce memory usage.

Listing 3.10: Whisper fine-tuning configuration

```
MODEL_NAME = "openai/whisper-medium"
processor = WhisperProcessor.from_pretrained(MODEL_NAME)
model = WhisperForConditionalGeneration.from_pretrained(MODEL_NAME)

forced_decoder_ids = processor.get_decoder_prompt_ids(lang="jp")
model.config.forced_decoder_ids = forced_decoder_ids
model.config.suppress_tokens = []
model.config.use_cache = False
```

3.6.3.4 Training Loop and Validation

Training was performed for 10 epochs using mini-batch gradient updates. During training, the loss value was monitored at each step and average loss was computed at the end of each epoch. After training, validation decoding was performed using `model.generate()` and metrics were computed using WER and CER. The model and processor were then saved for later decoding and test evaluation (Wolf et al., 2020).

3.6.3.5 Model Saving and Inference Check

After fine-tuning was completed, the model weights and processor configuration were saved using `save_pretrained`. A quick inference check was performed by decoding a sample from the test split and comparing the predicted transcription against the ground truth reference. This step was important to confirm that the model and tokenizer were saved correctly and could be loaded again for the evaluation phase (Wolf et al., 2020).

3.7 Result and Discussion Phase

Table 3.5 shows the fourth phase of this project where all trained models were evaluated using the same test split and the same scoring protocol and the result was interpreted and discussed in relation to the research questions and prior work. The main objective of this phase was to measure transcription accuracy and decoding ef-

ficiency in a consistent and reproducible way so that comparisons across the three model families were fair. This phase also discussed the trends observed across the three model families, explained the trade-offs between accuracy and speed, and outlined the limitations and future improvements. The activities carried out and the outcome deliverables are described in the table shown below.

Table 3.5 Result and Discussion Phase

Phase	Activities	Deliverables
Result and Discussion	Defined below items: • Scored with CER and WER • Measured speed using RTF • Compared models and decoders • Explained main findings • Linked to prior work • Noted limitations • Suggested future work	Objective 3 • Results tables and plots • Best model identified • Findings discussed • Limitations stated • Recommendations given • Future work proposed

In this phase, each trained system produced hypotheses on the test split and those hypotheses were scored against the same reference transcripts. The evaluation used three metrics that matched the research objectives, which are character error rate (CER), word error rate (WER), and real-time factor (RTF). Figure 3.7 shows the workflow for evaluating all three model families using the same test set and scoring protocol.

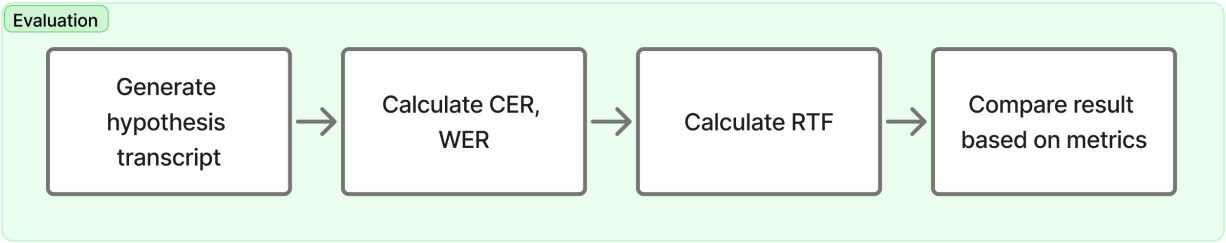

Figure 3.7 Evaluation workflow for all model families

3.7.1 Decoding and Hypothesis Generation

For the Kaldi GMM–HMM systems, decoding was performed using the test set features and the corresponding decoding graphs created earlier. Hypotheses were produced for each acoustic model stage (mono, tri1, tri2, tri3) using the standard

Kaldi decoding pipeline (Ghoshal et al., 2011). For the CRDNN–CTC system, decoding was performed using CTC decoding and hypotheses were generated for the test set, including the beam decoding variant when configured (Ravanelli et al., 2024). For Whisper variants, hypotheses were generated using the HuggingFace `model.generate()` decoding procedure, and the output token sequences were converted back into Japanese text using the Whisper tokenizer (Wolf et al., 2020).

3.7.2 Scoring Protocol (CER and WER)

After hypotheses were generated, each prediction was aligned against the reference transcripts and scored using CER and WER. CER was computed by measuring the normalized Levenshtein distance between predicted and reference character sequences. WER was computed using the same edit distance approach but applied at the word level after tokenization. The scoring protocol was applied consistently across all three model families so that differences in results reflected model performance rather than differences in evaluation handling (Jiwer, 2025).

Character Error Rate (CER) Let $\mathbf{r}_c$ be the reference character sequence and $\mathbf{h}_c$ be the hypothesis character sequence, with $N_c = |\mathbf{r}_c|$ reference characters. Then:

$$\mathrm{CER} = \frac{S_c + D_c + I_c}{N_c}. \tag{3.1}$$

Word Error Rate (WER) Let $\mathbf{r}_w$ be the reference word sequence and $\mathbf{h}_w$ be the hypothesis word sequence, with $N_w = |\mathbf{r}_w|$ reference words. Then:

$$\mathrm{WER} = \frac{S_w + D_w + I_w}{N_w}. \tag{3.2}$$

3.7.3 Speed Measurement (RTF)

Decoding speed was evaluated using real-time factor (RTF), defined as the ratio between total decoding wall time and total audio duration. Let T_{decode} denote the total wall-clock decoding time and T_{audio} denote the total duration of the evaluated audio. Then:

$$\mathrm{RTF} = \frac{T_{\text{decode}}}{T_{\text{audio}}}. \tag{3.3}$$

RTF values below 1.0 indicate faster-than-real-time decoding, while values above 1.0 indicate slower-than-real-time decoding.

After that, the results from CER, WER, and RTF were discussed to explain how each model family behaved under the same dataset and evaluation setup. The discussion compared classical and neural approaches by focusing on how performance improved from mono to tri3 in the Kaldi pipeline, how CRDNN–CTC handled Japanese character sequences under end-to-end training, and how Whisper variants achieved different accuracy and speed trade-offs depending on model size. The outcomes were linked back to prior work to show whether observed trends matched or differed from findings in the literature, especially regarding the advantages of pre-trained transformer-based models for low-resource or domain-mismatched settings (Bajo et al., 2024; Radford et al., 2023).

3.8 Summary

This chapter described the methodology for comparing Japanese ASR across three model families. The study followed four phases and used a single, consistent dataset pipeline to ensure fair comparison. Japanese TEDx talks with manual subtitles were collected, transcripts were cleaned, audio was standardised to 16 kHz mono, and long recordings were segmented using WebRTC VAD, with low-quality non-speech clips filtered out. All models were trained on the same 80/10/10 splits and evaluated on the same test set using CER, WER, and RTF to compare accuracy and decoding speed.

CHAPTER FOUR
RESULTS AND DISCUSSION

4.1 Introduction

In this chapter, the results of the workflow explained in Chapter 3 are presented. The results covered each step from data collection, preprocessing, model training, fine-tuning, and finally the metrics results for all tested models are presented. The performance is measured using character error rate (CER), word error rate (WER), and real-time factor (RTF), following the research objective explained in Chapter 1 and the gap identified in Chapter 2. The analysis in this chapter focuses on two goals. The first goal is to measure and compare the accuracy of the selected models, and the second goal is to measure how fast the models can generate output based on the input audio (Ando & Fujihara, 2021).

4.2 Data Collection and Preprocessing Results

4.2.1 Data Source Selection Outcomes

This study gathered data from publicly available Japanese TEDx talks and only videos that contained manual subtitles were selected from the playlist. Videos with auto-generated subtitles or subtitles in languages other than Japanese were excluded to reduce transcript noise.

Table 4.1 Video selection outcomes for TEDx Japanese data collection.

Item	Count
Number of videos from playlist	1574
Manual Japanese subtitles found	862
Auto-generated only / none	712
Download completed	862

Since the data source only utilized data from manual transcription, it aligned with the testing objective, which was to produce a readable and standardized transcript while at the same time reducing noise during training and evaluation. This was important because the Japanese language can have words with different meanings with

the same pronunciation or known as homophones. If this kind of data was used as training data, it would result in model performance becoming worse (Koenecke et al., 2020).

4.2.2 Transcript Cleaning and Normalization Outcomes

After the subtitles were downloaded, the raw data from the subtitles were normalized into a reference transcript that acted as the source of truth. This cleaning process involved removing formatting tags, removing speaker labels, and discarding non-speech annotations such as `[laughter]`, `(applause)`. The output from this process was a clean transcript with a timestamp. Another benefit of removing the labels was to reduce noise as things like bracket events would increase the error rate because when calculating the error number, the bracket event would be treated as a deletion from the reference transcript.

```
Timestamp: 00:00:01.000 --> 00:00:03.200
RAW      : [applause] みなさん、こんにちは。
CLEANED  : みなさん、こんにちは。

Timestamp: 00:00:03.200 --> 00:00:06.000
RAW      : Speaker 1: 今日はAIについて話します。
CLEANED  : 今日はAIについて話します。

Timestamp: 00:00:06.000 --> 00:00:09.000
RAW      : (笑) それでは始めましょう。
CLEANED  : それでは始めましょう。

Timestamp: 00:00:09.000 --> 00:00:12.000
RAW      : <i>重要なのは</i> 継続です。
CLEANED  : 重要なのは 継続です。

Timestamp: 00:00:12.000 --> 00:00:16.000
RAW      : [music] (拍手) 本日はありがとうございます。
CLEANED  : 本日はありがとうございます。
```

Figure 4.1 Raw vs cleaned transcript after subtitle cleaning and normalization

As shown in Figure 4.1, the raw subtitle contained non-speech items like formatting symbols and additional annotations that were useful for humans viewing the video but since they were not part of the intended script, they caused noise. Another thing to be taken into consideration was that the post-processing also needed to clean these kinds of annotations in the hypothesis transcript. This improved fairness when

comparing these models because the CER/WER was measured based on the speech content rather than subtitle formatting (Ando & Fujihara, 2021).

4.2.3 Audio Standardization Outcomes

After the audio was downloaded, it was converted into a consistent 16 kHz, mono, PCM16 format. The audio needed to be formatted to ensure compatibility when used in Kaldi, SpeechBrain, and Whisper training pipelines. By standardizing the sampling rate, it reduced the differences caused by audio encoders and also 16 kHz, mono, PCM16 is the standard data expected from traditional pipelines and modern neural pipelines (McFee et al., 2025). This step also supported the fair comparison goal stated in Chapter 1 by using the same data format across all models (C. Xu et al., 2023). Figure 4.2 below shows the difference between downloaded and resampled audio.

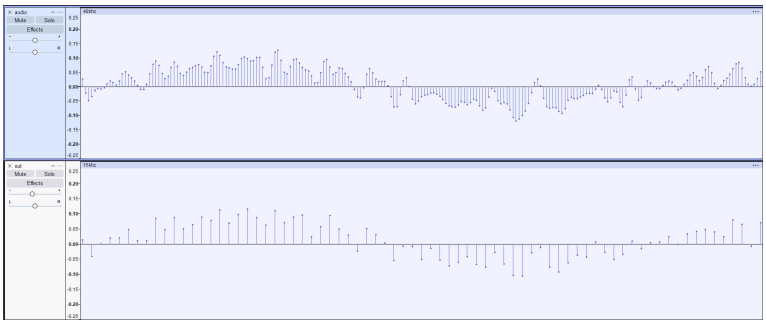

Figure 4.2 Example of audio standardization via resampling (44.1 kHz to 16 kHz).

4.2.4 VAD Segmentation Outcomes

The downloaded audio length was approximately 20 minutes long, which was not suitable for direct use as training data. By segmenting the long audio into shorter utterances, it was more practical to use as training data. Additionally, models like Whisper, which is based on transformers, could cause memory issues because long sequences of training data would increase memory usage and decrease the stability of the models during training and fine-tuning. The segmentation process was done using a lightweight segmentation mechanism widely used in speech preprocessing pipelines named WebRTC VAD (Blum et al., 2021).

Table 4.2 VAD segmentation results.

Item	Count / Value	Notes
Total chunks produced	74802	After segmentation and slicing
Chunks removed (too short)	11881	$\leq$2 s
Avg chunk duration (s)	6.605	After filtering short clips
Max chunk duration (s)	20	Enforced by slicing

Table 4.2 illustrates the summary of the data after segmentation. The average audio length became 6.6 seconds, which was closer to utterance-level speech than long-form audio. This eliminated the memory usage problem mentioned earlier, as the longest audio was only 20 seconds. Another reason to chunk the audio was to prevent the RTF measurement from creating a spike because of limits when decoding the audio.

4.2.5 Audio Filtering Outcomes

The filtering process was carried out based on the requirement in Chapter 1, which stated that the preprocessing method would heavily impact performance, especially for audio that contained too much noise, such as non-linguistic events or multi-speaker overlap (Latif et al., 2020). This type of data needed to be removed because it could contain weak or wrong subtitle alignment even when the segment had manual subtitles.

Table 4.3 Audio filtering outcomes.

Item	Count	Notes
Chunks before filtering	62921	Output from VAD
Removed by speaker-structure check	13758	Dominant speaker ratio threshold
Removed by clap/laughter threshold	5596	CLAP score thresholds
Chunks after filtering (final)	43567	Used to build manifests

The results in the table 4.3 illustrates that after removing non-speaker and low-quality audio, only 43567 chunks remained from 62921. This showed that it was a trade-off between the quantity and the quality of the audio, where filtering reduced the dataset size but increased the segments with clean Japanese speech. The reliability of CER/WER was also increased because the test reference was less affected by non-speech artifacts and misalignment noise (Ando & Fujihara, 2021).

4.2.6 Transcript-to-Audio Mapping and Manifest Outcomes

After the audio was segmented and cleaned, each chunk was paired with the reference transcript from the cleaned subtitles earlier. From there, the manifest containing the audio and transcript data was created. The objective of creating the manifest was to improve reproducibility when splitting the data into train and test splits. This also ensured that the differences in the end results were only due to the model itself and not due to inconsistent data splits (Ravanelli et al., 2021).

	ID	path	duration	transcript
0	1367_Why_a_city_...	/content/data/wa...	2.10	なぜそこの女川町を選んだのか。
1	398_Career_educa...	/content/data/wa...	3.69	あんなに堂々と 生き生きと発表す...
2	1250_１_TEDxWakay...	/content/data/wa...	3.81	こういう救出はこの到着部隊から考...
3	282_The_Regions_...	/content/data/wa...	10.89	これ人工衛星の話じゃないですか。...
4	1336_Bringing_a_...	/content/data/wa...	4.92	庭園 建物 様々なものづくりを発...
5	1348_A_face_to_f...	/content/data/wa...	3.90	たくさんの人を山に案内する中で ...
6	611_TEDxUSH_chun...	/content/data/wa...	3.99	何か自分の変わる人生が 変わるき...
7	1348_A_face_to_f...	/content/data/wa...	5.13	森は仕事として関わる一部の人間に...
8	1269_TEDxHaneda_...	/content/data/wa...	11.82	でも これって誰が悪いとか そう...
9	658_The_challeng...	/content/data/wa...	14.28	ハイビスカスをつけて 踊りながら...

Figure 4.3 Data after mapping audio and transcript into manifest format

The fig. 4.3 illustrates some of the data from the manifest data. The manifest data contained four columns which were ID, path, duration, and transcript. The ID data was used to differentiate each chunk from the main audio file name. Duration and path were the data of the audio file, keeping records of where the audio was stored and the duration of the audio file. Lastly, the transcript was the reference transcript that was used as labelled data when training models and as a reference transcript when testing the models.

4.2.7 Train/Validation/Test Split Results

The final dataset was split into 80% training, 10% validation, and 10% testing. Table 4.4 reported split sizes.

Table 4.4 Dataset split statistics.

Split	#Utterances	Duration (hrs)	Average (s)
Train	32852	63.3	6.941
Valid	4357	8.3	6.861
Test	4358	8.1	6.731

The split statistics show comparable average durations across train/valid/test, which reduces the risk that one split systematically contains longer or harder utterances. Keeping the test split fully held out supports a cleaner interpretation of generalization and makes the evaluation consistent with comparative reporting in prior Japanese ASR studies (Karita et al., 2021).

4.3 Model Training and Fine-Tuning Results

4.3.1 Kaldi GMM–HMM Training Outcomes

Kaldi training consisted of four stages of training, which were mono, tri1, tri2, and tri3. The later stages were trained based on the previous stages' aligned data. This convention followed the Kaldi recipe logic, where increasing the context-dependent model improved the alignment and feature-space transform. The consistent improvement aligned with established reports with other training recipes, such as CSJ, where later triphone and SAT produced better accuracy gains (Trmal, 2015).

Table 4.5 Kaldi GMM–HMM objective improvement summary.

Stage	Peak impr./frame	Peak iter	Final impr./frame	Stable iter (< 0.02)
mono	0.4289	1	0.0110	32
tri1	0.2022	3	0.0026	30
tri2	0.4820	1	0.0034	30
tri3	0.2975	3	0.0034	30

The trend line in Table 4.5 above shows that most of the substantial parameter updates occurred early in the training, then followed by small improvements until it hit a plateau and convergence was approached. This pattern was expected for GMM–HMM training and aligned with typical Kaldi diagnostics used to detect training stability and saturation (Ghoshal et al., 2011).

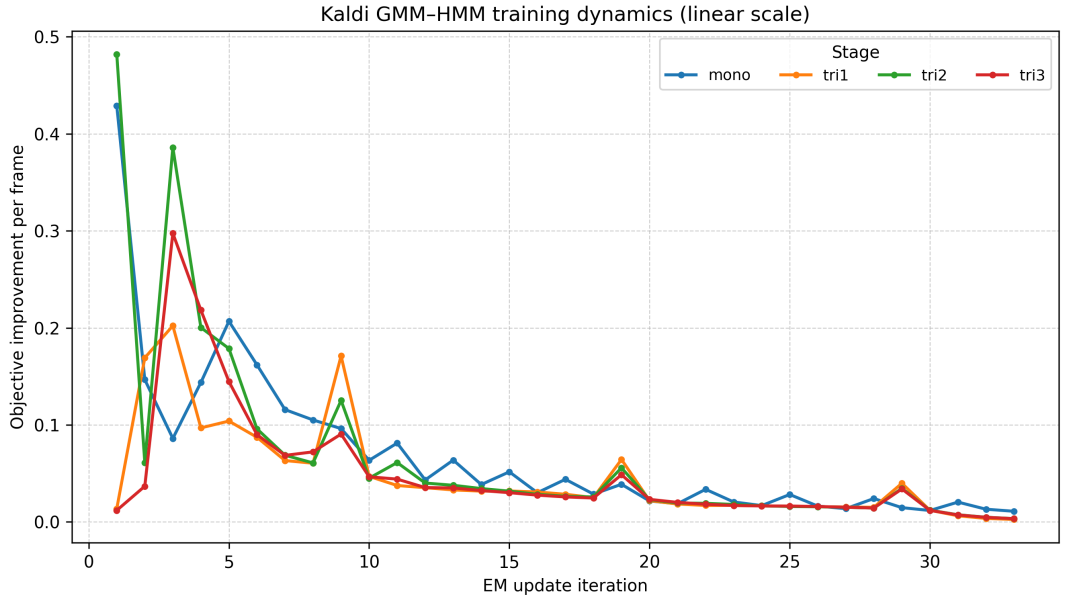

Figure 4.4 Kaldi GMM-HMM training dynamics in linear scale

4.3.2 CRDNN–CTC Training Outcomes

The CRDNN-CTC model was trained using the manifest prepared in the pre-processing phase and a character vocabulary derived from the training script. The training was done over 70 epochs, where the starting training loss was 5.44 at the start of training and became 0.0775 at the end, with the lowest CER at 0.2272 during epoch 68. After that, the best checkpoint was selected using validation tracking and later evaluated on the test split (reported in Table 4.10). Table 4.6 summarizes the epoch-by-epoch training dynamics from `data.txt`.

For the decoding part using CTC, it was well aligned with Japanese transcription due to its ability to use monotonic alignment without needing a pronunciation dictionary. The reference transcript also did not need to be tokenized into words, which was useful for languages that do not have spaces between words in a sentence (Watanabe et al., 2018).

Table 4.6 CRDNN–CTC training metrics across epochs.

Epoch	Train loss	Valid loss	CER
1	5.4400	5.2833	0.9983
2	4.0100	3.0769	0.6428
5	1.9100	1.6922	0.4555
10	1.1200	1.3025	0.3372
15	0.7760	1.0662	0.2906
20	0.5770	1.0610	0.2783
25	0.3130	1.0613	0.2629
30	0.2250	1.0297	0.2483
35	0.1740	1.0951	0.2404
40	0.1360	1.1158	0.2363
50	0.2050	1.0767	0.2437
60	0.1090	1.1674	0.2352
70	0.0775	1.2433	0.2309

Training curves can be observed from fig. 4.5 below showing that learning occurred very strongly in the early stages and then was followed by small changes later in the training as the epochs increased. The initial CER was 0.9983, indicating that the prediction was basically random at this point of training. As the train loss and valid loss decreased, the CER also decreased. The fig. 4.5 below shows that training and validation loss decreased as the epoch increased, with valid loss becoming flat first, followed by training loss.

Figure 4.5 CRDNN-CTC Training vs validation loss

fig. 4.6 below shows that CER decreased aggressively at the start of training and then became stable in the middle and maintained the trend until the end of training.

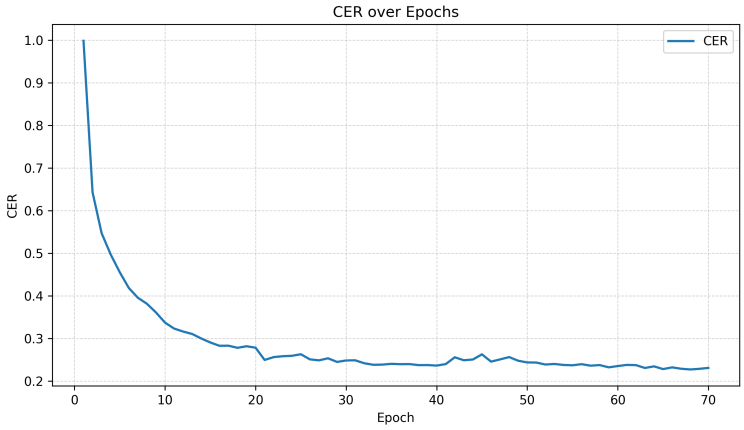

Figure 4.6 CRDNN-CTC CER over Epochs

4.3.3 Whisper Fine-Tuning Outcomes

The Whisper model fine-tuning used a pre-trained model provided by OpenAI, where the model was built upon a large-scale weak supervision, which has been shown to improve robustness under domain mismatch and reduce the reliance on task-specific feature engineering (Radford et al., 2023). Previous studies from Bajo et al. (2024) show that a notable improvement was achieved when fine-tuning the base multi-language models by adapting to the Japanese language. The fine-tune stability can be seen from the loss vs epoch values below.

Table 4.7 Whisper fine-tuning loss across epochs.

Epoch	tiny	base	small	medium	turbo
1	0.6626	0.3683	0.1988	0.0747	0.2137
2	0.3636	0.2106	0.0820	0.0415	0.1361
3	0.2862	0.1466	0.0413	0.0236	0.0911
4	0.2327	0.1034	0.0216	0.0130	0.0591
5	0.1930	0.0731	0.0115	0.0071	0.0362
6	0.1636	0.0520	0.0061	0.0035	0.0198
7	0.1426	0.0380	0.0033	0.0016	0.0095
8	0.1289	0.0296	0.0016	0.0012	0.0041
9	0.1205	0.0252	0.0008	0.0006	0.0016
10	0.1173	0.0234	0.0006	0.0003	0.0003

The larger models, like medium and large-turbo, became stable faster at only epoch 5, while the smaller models, like tiny, became stable after epoch 9. It is also worth noting that the loss value after epoch 10 was different even though the training

became stable, showing that the larger models were quicker to adapt to the training data compared to the smaller models.

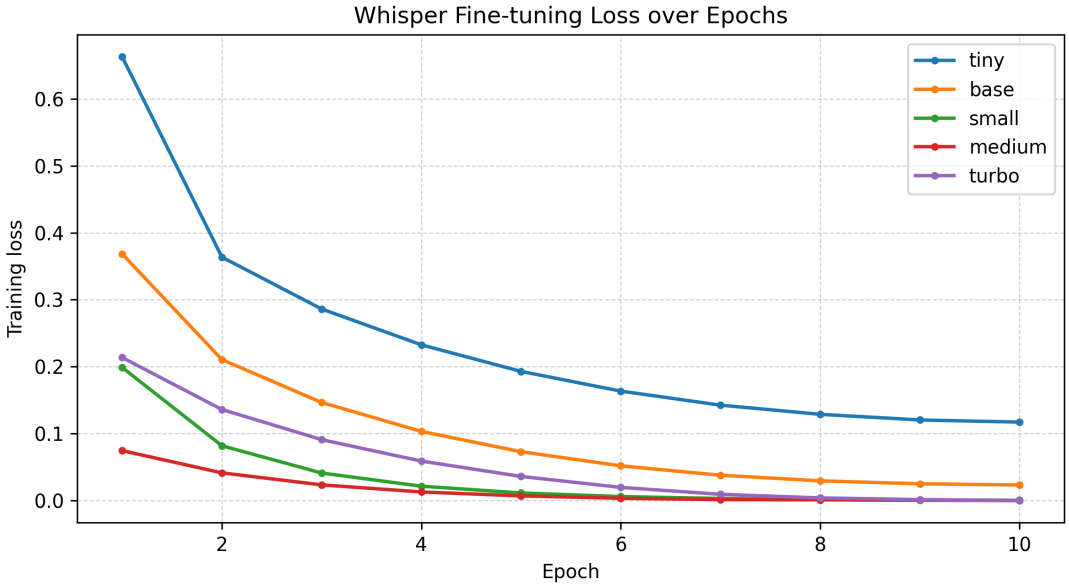

Figure 4.7 Whisper Fine-tuning loss over Epochs

4.4 Evaluation Results (CER, WER, and RTF)

4.4.1 Overall Results and Discussion

Table 4.8 below shows that the highest accuracy was achieved by fine-tuning the Whisper model by having the lowest CER and WER compared to the other two model families. However, from the aspect of decoding speed, CRDNN-CTC achieved the lowest RTF while having accuracy in second place after Whisper. The last model family was Kaldi, which showed good results and strong gains across training but was still less accurate compared to modern ASR approaches.

Table 4.8 Overall comparison of ASR systems on the test split (lower is better).

Model	CER (%)	WER (%)	RTF
GMMHMM-mono	49.49	53.43	0.6500
GMMHMM-tri1	24.95	29.31	0.3410
GMMHMM-tri2	21.30	25.59	0.2480
GMMHMM-tri3	19.48	23.57	0.2510
CRDNNCTC-1	15.23	22.87	**0.0129**
CRDNNCTC-2 (beam)	15.12	22.70	0.0147
Whisper-tiny	10.72	11.61	0.0297
Whisper-base	5.67	5.89	0.0413
Whisper-small	4.34	4.30	0.0737
Whisper-medium	4.29	**4.22**	0.1605
Whisper-turbo	**4.28**	4.23	0.0959

The pattern shown in the results aligned with the literature review in Chapter 2 where more recent end-to-end ASR models performed better compared to traditional models in terms of accuracy. However, when looking from the latency aspect, the architectural and decoding choices made differences as RTF for CRDNN-CTC was lower compared to Whisper (C. Xu et al., 2023). The reason was that CTC decoding was simpler and faster because it did not require an auto-regressive mechanism like Whisper, which predicted one token at a time based on previous tokens (Watanabe et al., 2018).

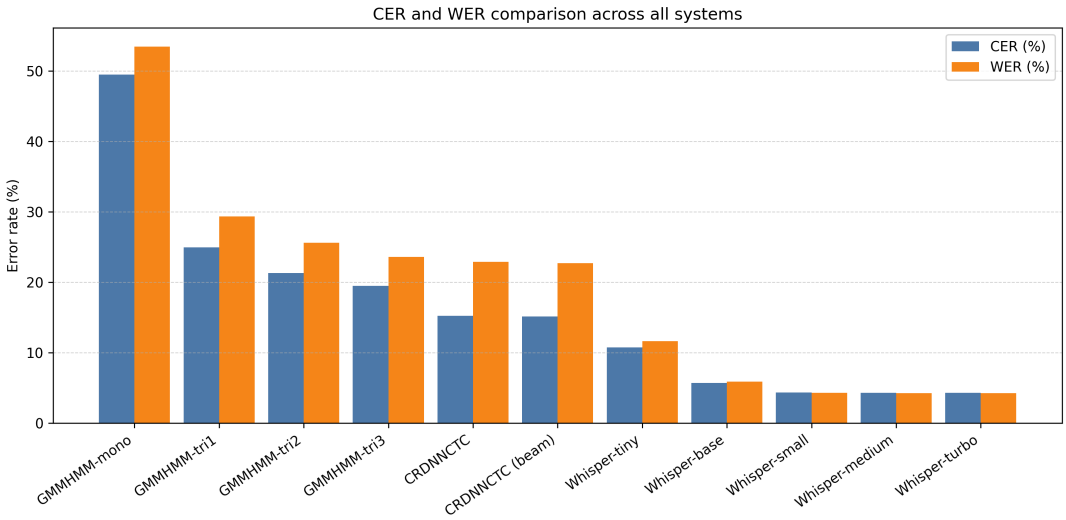

Figure 4.8 CER and WER comparison across all systems

The fig. 4.8 above shows that the trend between CER and WER was similar across all model families. One interesting pattern that can be observed is that

as the accuracy improved, the difference between CER and WER became smaller. Then starting from Whisper-small and above, the CER and WER almost converged. This was because the larger models had the ability to capture complex patterns better compared to smaller models. However, the difference was not very different after Whisper-small, showing that the model was already good enough to capture the patterns in the data.

Based on the results above, an observation can be made that the differences in accuracy and latency were affected by factors such as prior knowledge, decoding mechanism, and dependence on linguistic resources. Large pre-trained models like Whisper had advantages from large-scale pretraining on huge datasets which helped it to resolve ambiguous readings and maintain grammatical consistency even when acoustics were imperfect (Radford et al., 2023; C. Xu et al., 2023).

The nature of neural network architectures like CRDNN allowed the models to predict frame-level outputs directly from acoustic features, and this could eliminate the need for complex intermediate representations like phonemes or states used in GMM–HMM systems. This resulted in fewer resources to compute, took less time to decode, and produced lower RTF (Watanabe et al., 2018).

For traditional ASR models like Kaldi, they relied on feature engineering, lexicon/language modeling choices, and alignment quality which improved aggressively as more training stages were added. However, its modeling capacity was still limited compared to modern end-to-end systems because it could not leverage large-scale pretraining and flexible architectures which usually handled different data and domains better (Ando & Fujihara, 2021).

The overall outcomes aligned with the objective explained in Chapter 1 and with past studies reviewed in Chapter 2. ASR systems for Japanese speech recognition performed better when they handled symbol patterns and drew on broad pre-trained knowledge. Meanwhile, older processing methods still offered clearer logic paths and organized learning setups. However, their precision tended to drop when used across changing environments (Ando & Fujihara, 2021; Koenecke et al., 2020; C. Xu et al., 2023).

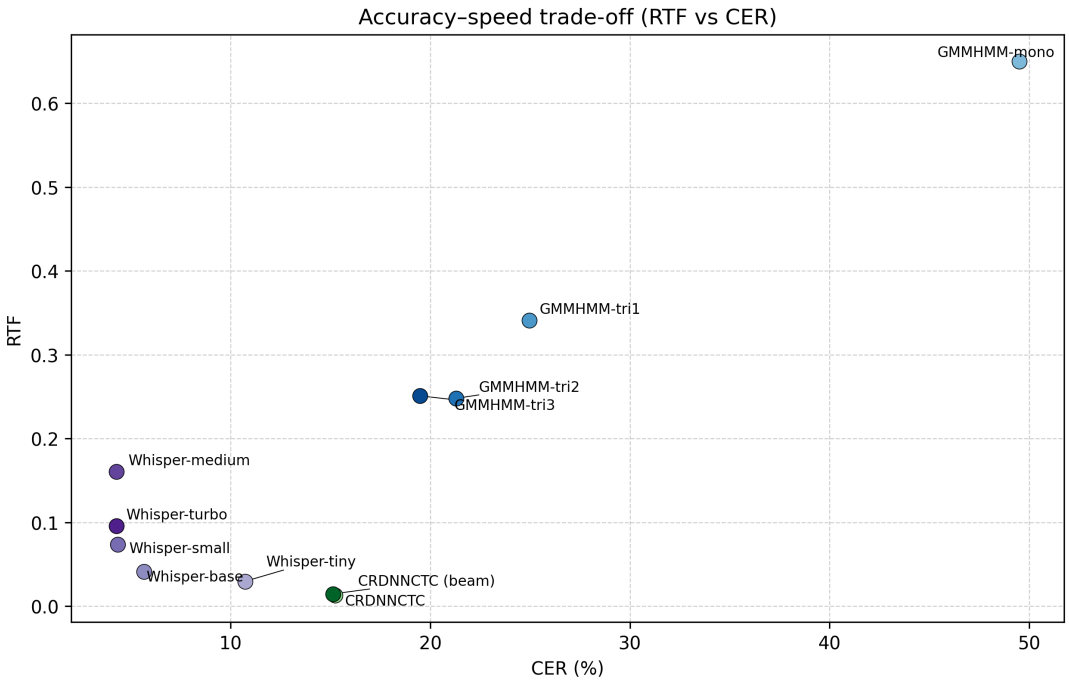

Figure 4.9 Accuracy speed trade-off

Figure 4.9 illustrates the trade-off between accuracy and speed across all models. The trend shows Whisper models dominated the accuracy aspect while maintaining a reasonable RTF. However, CRDNN-CTC was able to achieve a very low RTF while still having good accuracy.

4.4.2 Kaldi GMM–HMM Results and Discussion

The performance of Kaldi GMM-HMM consistently improved as the training stages advanced from mono to tri3. The first iteration using monophone models showed the highest error rate at 49.49% CER and 53.43% WER. Then after the model was aligned and trained using triphone models with delta features (tri1), the error rate dropped sharply to 24.95% CER and 29.31% WER. Further improvements were seen in tri2 with LDA+MLLT, reducing CER to 21.30% and WER to 25.59%. Finally, the speaker-adapted tri3 model achieved the best results with 19.48% CER and 23.57% WER. The biggest accuracy gain was between the mono and tri1 stages. After that point, each step only lowered the error rate by a small amount but continuously.

Table 4.9 Kaldi GMM–HMM results on the test split.

Stage	CER (%)	WER (%)	RTF
mono	49.49	53.43	0.6500
tri1	24.95	29.31	0.3410
tri2	21.30	25.59	0.2480
tri3	19.48	23.57	0.2510

The slower accuracy gains after tri1 were expected behaviour of these models where changing from monophones to context-dependent triphones brought significant improvements (Trmal, 2015). In terms of latency, the results show that the latency decreased from mono to tri2 where the RTF became stable from tri2 to tri3 even though the model was deeper.

The large increase in accuracy from mono to tri1 was expected because monophone models treated each phonetic unit independently while triphone models used context-dependent features. This was important in Japanese speech because there were many words that had the same pronunciation but different meaning depending on the context. By using triphones, the model could capture these contexts better (Ghoshal et al., 2011). By using triphone models, many substitution and deletion errors caused by context mismatch were reduced, explaining the steep initial drop in CER/WER.

From tri1 to tri3, the same modelling was used but with added feature transforms and speaker adaptation. This resulted in smaller gains compared to the initial switch to triphones but the improvement was consistent. LDA+MLLT in tri2 improved the model by improving feature-space discrimination while SAT in tri3 reduced speaker variability. These steps helped refine the acoustic modeling but could not fully overcome the inherent limitations of GMMs and reliance on pronunciation dictionaries and language models.

The RTF trends also showed the same pattern where the biggest drop was from mono to tri1, followed by a smaller drop to tri2, and then a slight increase from tri2 to tri3. This was because the more sophisticated models in tri2 and tri3 required additional computations for feature transforms and speaker adaptation and resulted in a slightly higher computational cost in tri3 compared to tri2.

The results shown from these models aligned with earlier studies where tra-

ditional Japanese ASR systems gained significantly from careful tuning of stages. However, the final accuracy score was still behind compared to advanced end-to-end models. A similar pattern also appeared in studies conducted by (Ando & Fujihara, 2021; Trmal, 2015).

4.4.3 CRDNN–CTC Results and Discussion

In the CRDNN-CTC results, when beam decoding was applied, the error rate for CRDNN-CTC dropped just below the greedy result. A shift from 22.87% down to 22.70% marked a small gap of 0.17%. However, the resources used in beam decoding were higher and resulted in the rise of RTF by roughly 13.95% over greedy methods. Despite minor accuracy gains, both decoding approaches produced the fastest decoding time compared to other models.

Table 4.10 CRDNN–CTC results on the test split.

Decoder	**CER (%)**	**WER (%)**	**RTF**
Greedy (CRDNNCTC-1)	15.23	22.87	**0.0129**
Beam (CRDNNCTC-2)	15.12	22.70	0.0147

The small difference in accuracy between the two decoding approaches suggested that the acoustic model carried more weight than the beam contribution. In this case, the performance leaned more on the encoder's strength rather than advanced decoding methods. Earlier studies also showed the same patterns where when language models added little, greedy strategies performed about the same as beam (Chen et al., 2020). The end result shows that both systems ran quickly, resulting in significantly low RTF and the speed made this model a practical choice for real-time applications where delay mattered.

The small difference between greedy and beam decoding suggested that in CRDNN-CTC, the model was strong enough to make good predictions without the need for additional sequence constraints. This also showed that most of the remaining error was likely due to acoustics which could not be improved using decoding strategies alone (Watanabe et al., 2018). This behaviour was expected in Japanese ASR because a lightweight LM may not strongly predict particles and words with only one or two characters.

The reason for the fast decoding in CRDNN-CTC was because of its architectural and decoding choices. The model did not rely on complex language models like GMM-HMM and Whisper, which required more computation during decoding. This model computed one output per frame directly from acoustic features, eliminating the need for multiple passes or rescoring steps. This structural difference directly explains why CRDNN–CTC reached the lowest RTF values among all tested systems (Watanabe et al., 2018).

4.4.4 Whisper Fine-Tuning Results and Discussion

Transcription accuracy reached its highest with Whisper fine-tuning. Moving up from tiny to base brought a clear improvement, where the WER dropped from 11.61% down to 5.89%. A further step to the small model reduced the WER again, landing at 4.30%. Progress beyond that slowed, where medium only edged ahead by 1.86%, lowering WER to 4.22%. At the same time, computational cost increased, and RTF climbed past 0.16. Yet, Whisper-turbo matched medium closely, hitting 4.23% WER while responding quicker than medium, with an RTF under 0.1.

Table 4.11 Whisper fine-tuning results on the test split.

Variant	**CER (%)**	**WER (%)**	**RTF**
tiny	10.72	11.61	0.0297
base	5.67	5.89	0.0413
small	4.34	4.30	0.0737
medium	4.29	**4.22**	0.1605
turbo	**4.28**	4.23	0.0959

The results reveal how speed tied to accuracy where bigger models gained precision only until gains slowed, demanding far more computing power. Earlier work supports this because pre-trained Transformers handled Japanese speech well, though size must suit real-world limits. Here, Whisper-turbo struck a useful middle ground, holding near top-tier word error rates while cutting response time compared to Whisper-medium.

The Whisper model showed significant accuracy improvements when scaling from tiny to base to small, showing that model capacity gave the model more ability to understand the context better. Better context understanding was very important in

Japanese because the language relied heavily on particles and context to differentiate the meaning of words (Radford et al., 2023).

However, the small improvement from small to medium suggested that a **ceiling effect** occurred under this dataset and evaluation setup. This was because remaining errors may be dominated by subtitle-reference mismatch, normalization differences, or ambiguous segments which could limit further gains from model size increases. This pattern aligned with prior findings where large pre-trained models reached diminishing returns on specific domains or languages (Bajo et al., 2024).

Whisper-turbo achieved near-medium accuracy with substantially lower RTF, suggesting it offered a better accuracy–latency balance for practical deployment. Given the small accuracy gap between medium and turbo (only 0.01 WER difference in Table 4.11) but a much lower RTF, turbo was a strong candidate when real-time or near-real-time constraints mattered, while medium remained best when maximum accuracy was prioritized. The reason for medium having higher WER but turbo having higher CER was because the turbo model was optimized for speed which may sacrifice some character-level accuracy while still maintaining word-level accuracy (Radford et al., 2023).

That the turbo model hit close to medium-level precision without slowing things down made it stand out. Since the drop in performance compared to medium was tiny, at just 0.01 on WER from Table 4.11, but it decoded faster. When getting results fast mattered more than perfection, turbo fit well. Turbo was a strong candidate when real-time or near-real-time constraints mattered, while medium remained best when maximum accuracy was prioritized.

The results were in line with the past study by Radford et al. (2023) where using a structured environment and a dataset like a TEDx event, Whisper was able to achieve good results because broad pre-trained systems were able to manage sound variation efficiently, especially once adapted through targeted training data. However, one thing that needed to be considered when choosing Whisper for real applications was that the resource usage was higher compared to other models.

4.5 Comparative Assessment

In this section the key comparative insights across the three ASR model families are summarized and highlighted based on accuracy, latency, and trade-off between the two.

4.5.1 Accuracy comparison (CER/WER)

Across all models evaluated, Whisper fine-tuned variants achieved the best transcription accuracy, with WER around 4.22% and CER around 4.28%. The next best performance came from CRDNN–CTC, which achieved mid-range accuracy of CER 15.12% and WER 22.70%. Traditional Kaldi GMM–HMM models trailed behind, with WER ranging from 23.57% (tri3) up to 53.43% (mono). This pattern suggests that pretrained encoder–decoder Transformers generalize better to TEDx speech and handle linguistic ambiguity more effectively than both conventional pipelines and smaller end-to-end encoders under the same data conditions (Radford et al., 2023; C. Xu et al., 2023).

4.5.2 Speed comparison (RTF) and deployment implications

For latency considerations, CRDNN–CTC models led the pack with the fastest decoding times, achieving RTF between 0.0129 and 0.0147. Whisper models exhibited increasing RTF with size, from tiny (0.0297) to medium (0.1605). Kaldi GMM–HMM models maintained real-time capability but lagged behind neural systems in accuracy. Overall RTF results reveal that if strict latency is required, CRDNN–CTC is preferred. However, if highest accuracy is required, Whisper is preferred while Kaldi remains valuable when interpretability and traditional resource control are priorities.

4.5.3 Accuracy–speed trade-off

Below is a summary of the accuracy–speed trade-off observed across the evaluated ASR systems:

- **High accuracy / medium speed:** Whisper medium and turbo provide the strongest recognition quality but require more computation.

- **Medium accuracy / High speed:** CRDNN–CTC balances good accuracy with very fast decoding, making it suitable for latency-sensitive applications.
- **Low accuracy / Low speed:** Kaldi models offer interpretability and traditional control but fall short in both accuracy and speed compared to modern neural approaches.

4.6 Error Analysis

The error analysis focused on qualitative review of transcription outputs from each ASR system on selected test utterances. Four representative examples were chosen to illustrate common error types, including wording errors, polite-form variations, paraphrasing, and numbering format inconsistencies. The reference transcripts were compared against outputs from Kaldi GMM–HMM (tri3), CRDNN–CTC (beam), and Whisper-turbo models. Differences were annotated using insertion, deletion, and substitution markers for clarity.

Although Whisper produced the fewest errors overall, the examples show it still made small mistakes such as minor polite-form variation or subtle substitutions. This supported the view that even strong pretrained models may require additional normalization rules or domain-specific constraints to achieve fully standardized outputs for archival or reporting use cases (Radford et al., 2023).

Table 4.12 Common error 1: Wording error.

System	Text / diff vs reference
Reference	来週、東京大学でAIの安全性について講演します。
GMM-HMM	来週[、→␣]東京大[-学-]でAIの安全[-性-]について[講→公]演します
CRDNN–CTC	来週、東京大学でAIの安全[-性-]について講演します
Whisper - turbo	来週、東京大学でAIの安全性について講演します

Table 4.13 Common error 2: Polite-form variation.

System	Text / diff vs reference
Reference	その結果をすぐに共有して、次の手順を決めましょう。
GMM-HMM	その結果[を→␣]すぐ[-に-]共有して[、→␣]次の手順[を→␣]決めましょう
CRDNN–CTC	その結果をすぐ[-に-]共有して、次の手順を決め[ま→よ][-し-][-ょ-]う
Whisper - turbo	その結果をすぐに共有して、次の手順を決め[ま→よ][-し-][-ょ-]う

Table 4.14 Common error 3: Paraphrase.

System	Text / diff vs reference
Reference	要するに、私たちは失敗から学ぶ必要があります。
GMM-HMM	要 す る に[、→␣]私 た ち は 失 敗 か ら 学 ぶ[必→␣][要→ひ][+つ+][+よ+][+う+]があります
CRDNN–CTC	[要→つ][す→ま][る→り][-に-]、私たちは失敗から学ぶ必要があります
Whisper - turbo	要するに、私たちは失敗から学ぶ必要があ[り→る][-ま-][-す-]

Table 4.15 Common error 4: Numbering format.

System	Text / diff vs reference
Reference	2024年にはクラウドへの移行が加速しました。
GMM-HMM	[2→二][0→千][2→二][4→十][+四+]年にはクラ[ウ→ン]ドへの移行が加速しました
CRDNN–CTC	2024年にはクラウドへの移行が加速しました
Whisper - turbo	2024年にはクラウドへの移行が加速しました

4.7 Impact of Preprocessing and Postprocessing Choices on Results

Because all systems used the same cleaned and standardized dataset, differences in accuracy and speed mainly reflected model architecture and decoding strategy rather than evaluation handling. Nevertheless, the preprocessing steps in Chapter 3 were designed to reduce transcript noise and non-speech audio, which supported stable training and fair comparison across model families (Ando & Fujihara, 2021; Latif et al., 2020).

A critical issue addressed during postprocessing was that when decoding for WER, it resulted in very high WER because Japanese text is often processed as a single continuous token for each sentence since it does not have spaces. So the strategy was to use tokenization to tokenize the sentence into words first using a Python library before calculating the WER.

Table 4.16 WER result before and after tokenization.

Model family	Model name	before	after
GMM–HMM	tri3	81.71%	23.57%
CRDNN–CTC	beam	77.74%	22.70%
Whisper	medium	6.82%	4.22%

Table 4.16 highlights the best model from each family with tokenized and non-tokenized WER. A clear pattern is that WER without tokenization was severely inflated for Japanese in the GMM–HMM and CRDNN–CTC systems, while tokenized WER became much lower and more interpretable. This happened because standard WER assumes space-delimited words. When Japanese sentences were evaluated without inserting word boundaries, an entire sentence could be treated as one token. In that case, even a small mistake could cause the whole sentence-token to be counted as a substitution, producing an unrealistically high WER. After tokenization, the same errors were distributed across multiple word tokens, so the score better reflected how much of the sentence was correct rather than penalizing the sentence as a single unit.

The smaller gap observed for Whisper ($6.82\% \rightarrow 4.22\%$) suggests that Whisper outputs were already closer to the reference in terms of overall lexical choice and normalization, so boundary decisions affected WER less. In contrast, GMM–HMM and CRDNN–CTC still produced more local errors that may interact strongly with the tokenizer's boundary decisions and therefore changed word-level scoring more dramatically. This finding supported the Chapter 1 motivation that Japanese WER must be reported with clearly documented tokenization for fair comparison across studies and systems (Ando & Fujihara, 2021). In this study, tokenized WER was treated as the primary WER metric, while no-tokenization WER was reported mainly to show how evaluation choices can distort conclusions for Japanese.

4.8 Limitations

Key limitations include TEDx-only speaking style, possible subtitle-to-speech mismatch, sensitivity of Japanese WER to tokenization, and limited ablation studies such as VAD thresholds, filtering thresholds, or language model variants. The TEDx domain emphasized relatively clear monologue speech, and therefore may not represent conversational Japanese or noisy real-world environments. In addition, subtitle references could still contain minor mismatches even when manually authored, which could place a ceiling on achievable CER/WER.

Another limitation in this study was the different resource requirements across model families. GMM–HMM model training and decoding were conducted on CPU, while CRDNN–CTC and Whisper required GPU acceleration for practical training

times. This difference may affect deployment considerations in resource-constrained settings. Another limitation was that only one beam width was tested for CRDNN–CTC beam decoding, and no language model variants were explored. Different beam widths or stronger language models could potentially improve accuracy further but were not evaluated here due to resource constraints. For Whisper, large model variants were not evaluated due to resource constraints and also for Whisper no training from scratch was not conducted due to computational limitations.

4.9 Summary

This chapter presented results for each stage of the Chapter 3 pipeline and compared ASR performance across classical and neural approaches. Kaldi improved substantially across staged training, CRDNN–CTC provided the fastest decoding, and fine-tuned Whisper variants achieved the best transcription accuracy with clear speed trade-offs by model size. The next chapter concludes the study and proposes future improvements.

CHAPTER FIVE
CONCLUSION AND RECOMMENDATION

5.1 Conclusion

This study aimed to fulfill three primary objectives: (1) to identify the key requirements for constructing a speech-to-text model within the context of the Japanese language; (2) to analyze speech-to-text models to determine the most effective preprocessing and training configurations for reducing CER, WER, and RTF; and (3) to evaluate the accuracy and latency of these models when transcribing Japanese speech.

5.1.1 Key Requirements for Japanese ASR (Objective 1)

To construct an effective speech-to-text model for Japanese, several critical requirements were identified and implemented. First, the audio had to be standardized into a consistent format (16 kHz, mono, PCM16) so that it could be used fairly across Kaldi, SpeechBrain, and Whisper. Second, raw downloaded audio had to be split into chunks to increase model stability and prevent memory errors. Third, noisy audio had to be filtered and cleaned to reduce transcript-to-audio mismatch. Finally, in postprocessing, tokenization was needed to calculate WER fairly. The successful identification and implementation of these steps addressed the first research objective.

5.1.2 Effective Pre-processing and Training Configurations (Objective 2)

This study analyzed three distinct model families under controlled preprocessing and evaluation conditions. The models used were the traditional Kaldi GMM–HMM, the hybrid CRDNN–CTC, and fine-tuned Whisper. For Kaldi, the most effective preprocessing was MFCC with CMVN and the training setup was staged training from mono into a triphone model. For CRDNN–CTC, the model was trained using the prepared manifest and evaluated using CTC decoding, including beam decoding for better output quality. For Whisper, multiple sizes were fine-tuned (tiny, base, small, medium, turbo) to observe the trade-off between accuracy and decoding speed. The analysis concluded that the fine-tuned Whisper architecture is the optimal setup for

transcription accuracy, whereas the CRDNN–CTC architecture is the most effective for low-latency applications, satisfying the second objective.

5.1.3 Performance Evaluation (Objective 3)

The quantitative evaluation demonstrated clear performance trade-offs between the model families. As shown in Chapter 4, fine-tuned Whisper achieved the best transcription accuracy, where the best WER was 4.22% (Whisper-medium) and the best CER was 4.28% (Whisper-turbo), with practical RTF values (Whisper-turbo RTF 0.0959 and Whisper-medium RTF 0.1605). CRDNN–CTC achieved the fastest decoding speed, with RTF around 0.013, but the accuracy was mid-range compared to Whisper (CER 15.12% and WER 22.70%). Kaldi GMM–HMM models were real-time capable but trailed behind neural models in accuracy, where the best GMM-HMM tri3 achieved WER 23.57%. In addition, the evaluation showed that Japanese WER is sensitive to tokenization, and without tokenization, WER can be inflated and misleading. These findings provide a comprehensive evaluation of accuracy and latency, fulfilling the third research objective.

5.2 Recommendation

For the recommendation and future work, this research project can be improved by using more diverse datasets to give better generalization, such as conversational Japanese, noisy recordings, overlapping speech, and different speaking styles. More experiments can be done by changing the preprocessing parameters such as the VAD aggressiveness levels, different filtering thresholds, and additional normalization rules to investigate their impact on transcription accuracy. For Kaldi–GMM, more advanced acoustic models can be explored to improve accuracy while maintaining real-time decoding. For CRDNN–CTC, stronger decoding settings can be explored, such as different beam widths and language model variants, because these can potentially reduce CER and WER. For Whisper, larger variants and additional domain adaptation can be evaluated if resources allow, and inference optimization such as quantization and streaming can be added to make training more efficient.

REFERENCES

Ando, S., & Fujihara, H. (2021). Construction of a large-scale japanese asr corpus on tv recordings. *ICASSP 2021 - 2021 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 6948–6952. https://doi.org/10.1109/ICASSP39728.2021.9413425

Baevski, A., Zhou, Y., Mohamed, A., & Auli, M. (2020). Wav2vec 2.0: A framework for self-supervised learning of speech representations. *Advances in neural information processing systems*, *33*, 12449–12460.

Bajo, M., Fukukawa, H., Morita, R., & Ogasawara, Y. (2024). Efficient adaptation of multilingual models for japanese asr. *arXiv preprint arXiv:2412.10705*.

Blum, N., Lachapelle, S., & Alvestrand, H. (2021). Webrtc. *Communications of the ACM*, *64*(8), 50–54. https://doi.org/https://doi.org/10.1145/3453182

Chen, J., Nishimura, R., & Kitaoka, N. (2020). End-to-end recognition of streaming japanese speech using ctc and local attention. *APSIPA Transactions on Signal and Information Processing*, *9*, e25. https://doi.org/10.1017/ATSIP.2020.23

Curtin, K. (2020). Japanese kanji power: A workbook for mastering japanese characters.

Futami, H., Ueno, S., Mimura, M., Sakai, S., Kawahara, T., et al. (2020). Rescoring hypotheses of automatic speech recognition with bidirectional transformer language model. *Proceedings of the 82nd National Convention of IPSJ*, *2020*(1), 175–176.

Ghoshal, A., Boulianne, G., Burget, L., Glembek, O., Goel, N., Hannemann, M., Motlíček, P., Qian, Y., Schwarz, P., Silovský, J., Stemmer, G., & Vesel, K. (2011). The kaldi speech recognition toolkit. *IEEE 2011 Workshop on Automatic Speech Recognition and Understanding*.

Graves, A., & Jaitly, N. (2014). Towards end-to-end speech recognition with recurrent neural networks. *International conference on machine learning*, 1764–1772.

Heafield, K. (2011). KenLM: Faster and smaller language model queries. In C.-B. Chris, P. Koehn, C. Monz, & O. F. Zaidan (Eds.), *Proceedings of the sixth*

workshop on statistical machine translation (pp. 187–197). Association for Computational Linguistics. https://aclanthology.org/W11-2123/

Hojo, N., Ijima, Y., & Mizuno, H. (2018). Dnn-based speech synthesis using speaker codes. *IEICE TRANSACTIONS on Information and Systems*, *101*(2), 462–472.

Hono, Y., Mitsuda, K., Zhao, T., Mitsui, K., Wakatsuki, T., & Sawada, K. (2024). Integrating pre-trained speech and language models for end-to-end speech recognition. https://arxiv.org/abs/2312.03668

Imaishi, R., & Kawabata, T. (2022). Examination of iterative estimation counts in gaussian mixture model-based speaker recognition. *Journal of the Acoustical Society of Japan*, *78*(11), 650–653.

Imaizumi, R., Masumura, R., Shiota, S., & Kiya, H. (2022). End-to-end japanese multi-dialect speech recognition and dialect identification with multi-task learning. *APSIPA Transactions on Signal and Information Processing*, *11*. https://doi.org/10.1561/116.00000045

Ito, H., Hagiwara, A., Ichiki, M., Mishima, T., Sato, S., & Kobayashi, A. (2016). End-to-end neural network modeling for japanese speech recognition. *Journal of the Acoustical Society of America*, *140*, 3116–3116. https://doi.org/10.1121/1.4969755

Ito, H., Hagiwara, A., Ichiki, M., Mishima, T., Sato, S., & Kobayashi, A. (2017). End-to-end speech recognition for languages with ideographic characters. *2017 Asia-Pacific Signal and Information Processing Association Annual Summit and Conference (APSIPA ASC)*, 1228–1232. https://doi.org/10.1109/APSIPA.2017.8282226

Jiwer. (2025, June). https://pypi.org/project/jiwer/

Karita, S., Kubo, Y., Bacchiani, M. A. U., & Jones, L. (2021). A comparative study on neural architectures and training methods for japanese speech recognition. *Proceedings of the Annual Conference of the International Speech Communication Association, INTERSPEECH*, *2*, 2092–2096. https://doi.org/10.21437/INTERSPEECH.2021-775

Koenecke, A., Nam, A., Lake, E., Nudell, J., Quartey, M., Mengesha, Z., Toups, C., Rickford, J. R., Jurafsky, D., & Goel, S. (2020). Racial disparities in automated speech recognition. *Proceedings of the National Academy of Sciences*

of the United States of America, *117*, 7684–7689. https://doi.org/10.1073/PNAS.1915768117/SUPPL_FILE/PNAS.1915768117.SAPP.PDF

Kohei Mukohara, S. N., Koichiro Yoshino, et al. (2015). Investigation of dnn and cnn bottleneck features in emotional speech recognition. *Research Report on Speech and Language Processing (SLP)*, *2015*(15), 1–6.

Kolesnikov, A., Beyer, L., Zhai, X., Puigcerver, J., Yung, J., Gelly, S., & Houlsby, N. (2020). Big transfer (bit): General visual representation learning. *Computer Vision–ECCV 2020: 16th European Conference, Glasgow, UK, August 23–28, 2020, Proceedings, Part V 16*, 491–507.

Kubo, Y. (2014). Deep learning for speech recognition (series explanation: Deep learning [part 5]). *Artificial Intelligence*, *29*(1), 62–71.

Latif, S., Qadir, J., Qayyum, A., Usama, M., & Younis, S. (2020). Speech technology for healthcare: Opportunities, challenges, and state of the art. *IEEE Reviews in Biomedical Engineering*, *14*, 342–356.

Lin, S., Tsunakawa, T., Nishida, M., & Nishimura, M. (2017). Dnn-based feature transformation for speech recognition using throat microphone. *2017 Asia-Pacific Signal and Information Processing Association Annual Summit and Conference (APSIPA ASC)*, 596–599.

McFee, B., McVicar, M., & and, D. F. (2025, March). *Librosa* (Version 0.11.0). Zenodo. https://doi.org/10.5281/zenodo.15006942

Mu, D., Sun, W., Xu, G., & Li, W. (2020). Japanese pronunciation evaluation based on ddnn. *IEEE Access*, *8*, 218644–218657.

Noda, K., Yamaguchi, Y., Nakadai, K., Okuno, H. G., Ogata, T., et al. (2014). Lipreading using convolutional neural network. *Interspeech*, *1*, 3.

Povey, D., Burget, L., Agarwal, M., Akyazi, P., Kai, F., Ghoshal, A., Glembek, O., Goel, N., Karafiát, M., Rastrow, A., et al. (2011). The subspace gaussian mixture model—a structured model for speech recognition. *Computer Speech & Language*, *25*(2), 404–439.

Pykakasi. (2022). https://pykakasi.readthedocs.io/en/latest/index.html

PySoundFile. (2015). Pysoundfile. https://python-soundfile.readthedocs.io/en/0.13.1/

Python, S. F. (2025). Python documentation. https://docs.python.org/3/

Radford, A., Kim, J. W., Xu, T., Brockman, G., McLeavey, C., & Sutskever, I. (2023). Robust speech recognition via large-scale weak supervision. *International conference on machine learning*, 28492–28518.

Radford, A., Wu, J., Child, R., Luan, D., Amodei, D., Sutskever, I., et al. (2019). Language models are unsupervised multitask learners. *OpenAI blog*, *1*(8), 9.

Ravanelli, M., Parcollet, T., Moumen, A., de Langen, S., Subakan, C., Plantinga, P., Wang, Y., Mousavi, P., Libera, L. D., Ploujnikov, A., Paissan, F., Borra, D., Zaiem, S., Zhao, Z., Zhang, S., Karakasidis, G., Yeh, S.-L., Champion, P., Rouhe, A., … Estève, Y. (2024). Open-source conversational ai with speechbrain 1.0. *Journal of Machine Learning Research*, *25*(333). http://jmlr.org/papers/v25/24-0991.html

Ravanelli, M., Parcollet, T., Plantinga, P., Rouhe, A., Cornell, S., Lugosch, L., Subakan, C., Dawalatabad, N., Heba, A., Zhong, J., Chou, J.-C., Yeh, S.-L., Fu, S.-W., Liao, C.-F., Rastorgueva, E., Grondin, F., Aris, W., Na, H., Gao, Y., … Bengio, Y. (2021). Speechbrain: A general-purpose speech toolkit. https://arxiv.org/abs/2106.04624

Rawat, A., Rawat, V., Singh, N., Kuchhal, N., Barmola, J., & Negi, H. S. (2023). An enhanced version of youtube video downloader using python. *2023 International Conference on Computer Science and Emerging Technologies (CSET)*, 1–6.

Rose, H. (2019). Unique challenges of learning to write in the japanese writing system. *L2 writing beyond English*, *66*.

Seki, H., Yamamoto, K., & Nakagawa, S. (2014). Comparison of syllable-based and phoneme-based dnn-hmm in japanese speech recognition. *2014 International Conference of Advanced Informatics: Concept, Theory and Application (ICAICTA)*, 249–254.

Sun, R. H., & Chol, R. J. (2020). Subspace gaussian mixture based language modeling for large vocabulary continuous speech recognition. *Speech Communication*, *117*, 21–27.

Takahashi, N., Miwa, S., Kamiya, Y., Toyama, T., Nahar, R., & Kai, A. (2024). Comparison of large pre-trained models and adaptation methods for japanese

dialects asr. *2024 IEEE 13th Global Conference on Consumer Electronics (GCCE)*, 811–814.

Takami, J., & Kawabata, T. (2020). Speaker recognition performance metric for small-scale voice dialogue systems based on adaptation speed and convergence accuracy of gaussian mixture models. *Journal of the Acoustical Society of Japan*, *76*(5), 254–261.

Takeuchi, D., Yatabe, K., Koizumi, Y., Oikawa, Y., & Harada, N. (2020). Real-time speech enhancement using equilibrated rnn. *ICASSP 2020-2020 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 851–855.

Taniguchi, S., Kato, T., Tamura, A., & Yasuda, K. (2022). Transformer-based automatic speech recognition with auxiliary input of source language text toward transcribing simultaneous interpretation. *INTERSPEECH*, 2813–2817.

Taniguchi, S., Kato, T., Tamura, A., Yasuda, K., et al. (2024). Pre-training of transformer-based asr for simultaneous interpretation with auxiliary input of source language text using large machine translation corpus. *Proceedings of the 86th National Convention of IPSJ*, *2024*(1), 397–398.

Tokuda, K. (1999). Application of hidden markov models to speech synthesis. *IEICE Technical Report, SP99-61*, 48–54.

Tokuda, K. (2000). Fundamentals of speech synthesis using hmm. *IEICE Technical Report, SP2000-74*, 43.

Trmal, J. (2015). Kaldi CSJ recipe results (egs/csj/s5/results) [Accessed: 2025-12-29].

Watanabe, S., Hori, T., Karita, S., Hayashi, T., Nishitoba, J., Unno, Y., Yalta Soplin, N. E., Heymann, J., Wiesner, M., Chen, N., Renduchintala, A., & Ochiai, T. (2018). Espnet: End-to-end speech processing toolkit. *Interspeech 2018*, 2207–2211. https://doi.org/10.21437/Interspeech.2018-1456

Watanabe, S., Hori, T., Kim, S., Hershey, J. R., & Hayashi, T. (2017). Hybrid ctc/attention architecture for end-to-end speech recognition. *IEEE Journal of Selected Topics in Signal Processing*, *11*(8), 1240–1253. https://doi.org/10.1109/JSTSP.2017.2763455

Wolf, T., Debut, L., Sanh, V., Chaumond, J., Delangue, C., Moi, A., Cistac, P., Rault, T., Louf, R., Funtowicz, M., Davison, J., Shleifer, S., von Platen, P., Ma,

C., Jernite, Y., Plu, J., Xu, C., Scao, T. L., Gugger, S., ... Rush, A. M. (2020). Huggingface's transformers: State-of-the-art natural language processing. https://arxiv.org/abs/1910.03771

Wu, Y., Chen, K., Zhang, T., Hui, Y., Berg-Kirkpatrick, T., & Dubnov, S. (2022). Large-scale contrastive language-audio pretraining with feature fusion and keyword-to-caption augmentation. https://doi.org/10.48550/ARXIV.2211.06687

Xu, C., Ye, R., Dong, Q., Zhao, C., Ko, T., Wang, M., Xiao, T., & Zhu, J. (2023). Recent advances in direct speech-to-text translation. *arXiv preprint arXiv:2306.11646*.

Xu, W., Dainoff, M. J., Ge, L., & Gao, Z. (2023). Transitioning to human interaction with ai systems: New challenges and opportunities for hci professionals to enable human-centered ai. *International Journal of Human–Computer Interaction*, *39*(3), 494–518. https://doi.org/10.1080/10447318.2022.2041900

Yalta, N., Watanabe, S., Hori, T., Nakadai, K., & Ogata, T. (2019). Cnn-based multichannel end-to-end speech recognition for everyday home environments. *2019 27th European Signal Processing Conference (EUSIPCO)*, 1–5.

Yusuke Kida, T. T., et al. (2016). Reverberant speech recognition based on linear predictive filter estimation using lstm. *Research Report on Speech and Language Processing (SLP)*, *2016*(25), 1–6.